



Building AI Agents



General Course Remarks

- Course times
- Drinks
- Meals
- Toilettes
- Smoking
- Mobile Phones
- Manual
- Software

Everything should be made as simple as possible, but not simpler

Albert Einstein

Ja, was ich hier geschrieben habe, macht im Einzelnen überhaupt nicht den Anspruch auf Neuheit; und darum gebe ich auch keine Quellen an, weil es mir gleichgültig ist, ob das was ich gedacht habe, vor mir schon ein anderer gedacht hat.

Ludwig Wittgenstein

► Intro AI Agents

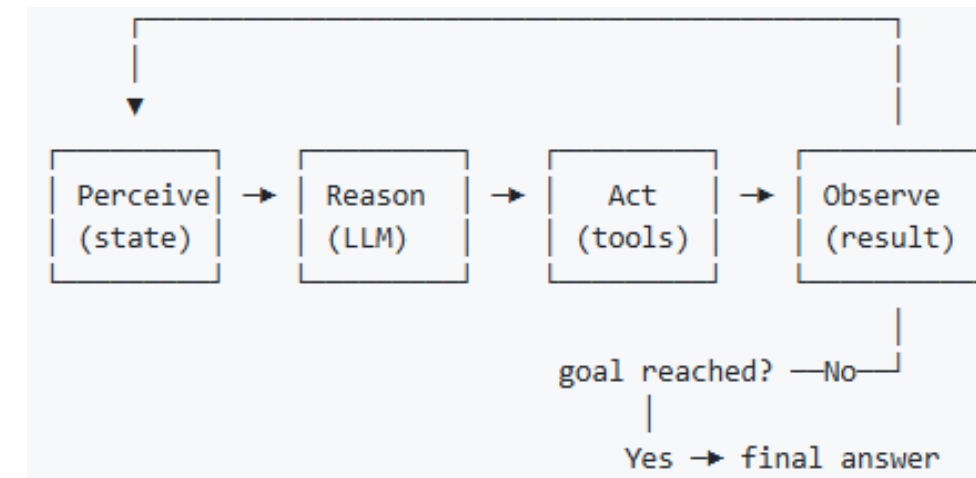
- LLM Fundamentals
 - LangChain Intro
 - LangChain Components
 - Agents and Tools
 - LangGraph Intro
 - Multi Agent Systems
 - Model Context Protocol
 - Running Agents
- What are AI Agents?
 - Basic Agent Cycle
 - Agents versus Workflows
 - Running the Labs
 - Why Agents Became Prominent?
 - Criteria to Build Agents
 - Agent Design Foundations
 - Selecting Models
 - Defining Tools
 - Types of Tools
 - Agent Design Patterns
 - Agent Frameworks and Libraries
 - LangChain
 - LangGraph
 - OpenAI Agents SDK
 - Using OpenAI Agents SDK
 - Claude Agent SDK
 - CrewAI
 - Other Frameworks
 - Configuring Instructions

What are AI Agents?

- **An AI agent is software built around a large language model :**
 - The model reasons about a goal and decides what to do about it
- **The agent uses tools to carry those decisions out :**
 - It observes what came back, and keeps going until the goal is met
- **All of that happens with little or no step-by-step human direction**
- **A plain LLM call maps one input to one output :**
 - It answers a single prompt and stops there
- **An agent runs a loop instead :**
 - It can call a calculator, query a database, search the web or write a file
 - Then it reads the result and decides what to do next
- **The definition worth remembering :**
 - An agent is an LLM in a control loop, equipped with tools and memory, that pursues a goal by repeatedly deciding what to do next

Basic Agent Cycle

- Every framework runs the same four-step cycle
- **Perceive :**
 - The request, the previous steps, the tool results and the relevant memory
- **Reason :**
 - The model interprets the situation and plans the next step
- **Act :**
 - Usually a tool call ; a search, an API request, code execution, a query
- **Reflect :**
 - Read the result and decide whether the goal has been reached
- The loop repeats until a final answer, or until a stopping condition fires



Agents versus Workflows

- In a workflow the developer fixes the path through the code
- In an agent the model chooses the path at run time
- Most production systems are a hybrid :
 - A fixed workflow, with agentic steps at the points that are uncertain
- Write a workflow when the steps can be hard-coded :
 - It is cheaper, faster and far easier to debug
- Reach for an agent when the steps depend on intermediate results

System type	Who decides the steps?	Example
Single LLM call	The developer (one prompt → one answer)	"Summarise this paragraph."
Chatbot	The developer, turn by turn	A Q&A assistant that answers each message in isolation.
Workflow / chain	The developer, in advance (fixed steps)	"Retrieve docs → summarise → translate." Always the same path.
Agent	The LLM, at run time	"Research this topic and write a report" — the model chooses which tools to call and how many times.

exercise26

Which Tasks Need an Agent

Running the Labs

- **Every demo and every exercise is a notebook, and nothing is installed :**
 - In the browser, through Google Colab, on any laptop
 - Locally through Ollama, if we would rather keep it on our own machine
- **The same notebooks serve both, so neither path is the cut-down one**
- **Each one opens with the same bootstrap cell, which picks the provider :**

```
colab    hosted course model, key from Colab Secrets    (the default here)
ollama   local model, no key, no cost
```

- **`make_llm()` and `make_embeddings()` hide the difference :**
 - Nothing below that cell is provider-specific
- **Setup is in `notebooks/SETUP-COLAB.md` , or `SETUP-LOCAL.md` to run offline**

Why Agents Became Prominent?

- **Three ingredients had to mature before agents became practical**
- **Tool use, also called function calling :**
 - The model reliably emits a structured request to call an external function
 - It then uses the data that comes back
- **Larger and cheaper context windows :**
 - Room for the instructions, the tool results and the history the loop accumulates
- **Stronger reasoning :**
 - Models good enough to plan a multi-step task, and to recover from a mistake
- **Through 2025 and 2026 the industry's attention moved on :**
 - Away from "which model is best?"
 - Towards "what does the harness around the model need to do?"
- **Most of the engineering effort now goes into the harness around the model**

Criteria to Build Agents

- **Three signals say the problem is worth an agent**
- **Complex decision-making :**
 - Work involving nuanced judgment, exceptions or context-sensitive decisions
 - Approving a refund in a customer service workflow is the standard example
- **Rules that have become hard to maintain :**
 - A ruleset grown so large that every update is costly or error-prone
 - Vendor security reviews are a common case
- **Heavy reliance on unstructured data :**
 - Interpreting natural language, reading documents, talking to users
 - Processing a home insurance claim needs all three
- **Check the use case against these before committing :**
 - A deterministic solution is cheaper whenever one will do

Agent Design Foundations

- In its most basic form an agent has three components
- The model does the reasoning and makes the decisions
- The tools are the external functions and APIs it can call to act
- The instructions are the guidelines and guardrails that shape its behaviour
- Written with the OpenAI Agents SDK :

```
agent = Agent(  
    name="Weather Bot",  
    instructions="You are a helpful weather assistant. Use tools to get weather data.",  
    model="gpt-5.6-luna",  
    tools=[get_weather, get_forecast]  
)
```

- Every other framework in this course expresses the same three components

Selecting Models

- **Models differ in the task complexity they handle, in latency and in cost**
- **Not every task needs the strongest model :**
 - Retrieval and intent classification suit a smaller, faster one
 - Approving a refund benefits from a more capable one
- **Mixing models across one workflow is normal, and is called orchestration**
- **Establish a baseline before optimising :**
 - Prototype with the most capable model for every task
 - Then swap in smaller models and measure where they still succeed
- **Three principles :**
 - Set up evaluations to establish a baseline
 - Meet the accuracy target with the best model available
 - Then cut cost and latency by substituting smaller models

Defining Tools

- **Tools extend the agent by exposing the APIs of the systems around it**
- **Legacy systems without an API can still be reached :**
 - Computer-use models drive the web and desktop interface as a human would
- **Give every tool a standardised definition :**
 - Any agent can then use any tool, without a bespoke wrapper per pairing
- **Document and test tools properly :**
 - It improves discoverability, helps version management and prevents duplicates
- **Agents need three kinds of tool :**
 - Data tools retrieve information
 - Action tools change something in an external system
 - Orchestration tools coordinate other tools, and other agents

Types of Tools

Type	Description	Examples
Data	Enable agents to retrieve context and information necessary for executing the workflow.	Query transaction databases or systems like CRMs, read PDF documents, or search the web.
Action	Enable agents to interact with systems to take actions such as adding new information to databases, updating records, or sending messages.	Send emails and texts, update a CRM record, hand-off a customer service ticket to a human.
Orchestration	Agents themselves can serve as tools for other agents—see the Manager Pattern in the Orchestration section.	Refund agent, Research agent, Writing agent.

Agent Design Patterns

- A set of recurring patterns has emerged over the past few years
- **Single-agent patterns :**
 - ReAct, reason and act ; the agent interleaves thinking and acting in a loop
 - Plan-and-execute ; the agent writes an explicit plan, then runs the steps
 - Reflection ; the agent judges its own work against criteria and revises it
 - RAG agent ; the agent's main job is pulling in external knowledge
 - Human-in-the-loop ; the agent pauses for review at defined points
- **Multi-agent patterns :**
 - Supervisor ; one agent splits the job and delegates to specialist sub-agents
 - Handoff ; agent A transfers control of the conversation to agent B
 - Conversational ; several agents talk to each other in a shared conversation
 - Agent-as-tool ; one agent is exposed to another as an ordinary tool
- **Modules 05 to 07 build examples of most of these**

Agent Frameworks and Libraries

- The versions here are a September 2026 snapshot ; check them before relying on them
- **Provider-native SDKs give deep integration and less model flexibility :**
 - OpenAI Agents SDK, Claude Agent SDK, Google ADK
- **Model-agnostic frameworks work across providers :**
 - LangChain and LangGraph, CrewAI, PydanticAI, LlamaIndex
- **Enterprise and .NET oriented :**
 - Microsoft Agent Framework, which absorbed AutoGen and Semantic Kernel
- **Low-code and visual platforms :**
 - Dify, n8n and Flowise build agents in a UI rather than in code
- **Neither of the first two categories wins outright :**
 - The trade is depth of integration against freedom to change model

LangChain

- **The broadest and most widely adopted toolkit for LLM applications and agents :**
 - It puts a provider-agnostic interface over models, tools and agents
 - Available for Python and for JavaScript and TypeScript
 - **It reached v1.0 in October 2025 and has been on the 1.x line since :**
 - The v1 rewrite refocused the library on the core agent loop
 - `create_agent` replaced the older agent constructors
 - Middleware was added for customising agent behaviour
 - **The ecosystem carries over 1,000 integrations**
- ```
pip install langchain langchain-openai
```
- **The best choice for starting quickly and swapping providers freely**



# LangGraph

- **A lower-level orchestration framework and runtime from the LangChain team :**
  - An agent is a state graph ; the nodes are actions and the edges are transitions
  - We get precise control over branching, loops, retries and human checkpoints
- **Available for Python and for JavaScript and TypeScript, on the 1.x line**
- **Execution is durable :**
  - The agent survives a server restart and continues where it left off
- **Persistence, human-in-the-loop and streaming are built in**
- **The best choice for complex, stateful, long-running and multi-agent workflows**

# OpenAI Agents SDK

- **A provider-native SDK for orchestrating multi-agent systems :**
  - The primitives are agents, handoffs, tools, guardrails, sessions and tracing
  - It is OpenAI-native but model-flexible, and supports many other providers
- **Available for Python and TypeScript**
- **April 2026 added a model-native harness :**
  - File operations and code execution
  - Native sandbox execution across several sandbox providers
- **The best choice for explicit multi-agent coordination through handoffs :**
  - It is also the strongest option for voice and realtime speech agents
  - It suits teams that want few abstractions between them and the model

# Using OpenAI Agents SDK

- An agent is equipped with a list of tools :

```
from agents import Agent, WebSearchTool, function_tool

@function_tool
def save_results(output):
 db.insert({"output": output, "timestamp": datetime.time()})
 return "File saved"

search_agent = Agent(
 name="SearchAgent",
 instructions="Help user search the internet and save results if asked.",
 tools=[WebSearchTool(), save_results],
)
```

- Split the work across several agents as the tool count grows :
  - Coordinating them is what the SDK calls orchestration

# Claude Agent SDK

- Exposes the agent loop, the tools and the context management behind Claude Code :
  - The minimal agent is the async `query()` primitive plus `ClaudeAgentOptions`
  - The SDK owns the tool loop, file access, bash, web search and compaction
  - It was renamed from Claude Code SDK in 2026
- Available for Python and TypeScript
- Deep MCP integration, lifecycle hooks, subagents and structured output
- The best choice for coding agents and for workflows that need real OS access

# CrewAI

- A role-based multi-agent framework, Python only
- We define a crew rather than a graph :
  - Every agent gets a role, a persona and its own tools
  - Tasks are assigned, and CrewAI orchestrates who does what
  - Researcher to writer to reviewer is the canonical shape
- The fastest route from an idea to a working multi-agent prototype
- MCP is supported natively through `crewai-tools`
- The abstraction is also the tradeoff :
  - Intuitive while the work fits a crew of roles
  - Limiting once the control flow needs to branch or loop in its own way

# Other Frameworks

- **Microsoft Agent Framework :**
  - Reached 1.0 in April 2026, merging AutoGen and Semantic Kernel
  - The path Microsoft now points new work at ; Python and C#
  - AutoGen itself is in maintenance mode and gets no new features
- **Google ADK :**
  - Orchestrates a hierarchical tree of agents
  - Agent Cards let agents built on different frameworks interoperate
  - Available for Python, Java and Go
- **PydanticAI :**
  - Type safety and schema-validated output, built on Pydantic
- **LlamaIndex :**
  - A RAG data framework grown into a knowledge-aware agent platform

# Configuring Instructions

- **Instructions reduce ambiguity and drive the agent's decisions**
- **Start from documents that already exist :**
  - Operating procedures, support scripts, knowledge base articles
- **Break dense material into smaller steps :**
  - Each step maps to one specific action or output
  - Ask for an order number, look up an account, send a reply
- **Being explicit about the action leaves less room for misinterpretation**
- **Capture the edge cases :**
  - Name the common variations and say how each is handled
  - Incomplete information is the most frequent decision point of all

# Summary : Intro AI Agents

- Models now handle complex, multi-step tasks
- An AI agent is an LLM in a control loop with tools and memory
- The agent uses the model to manage its own workflow and decisions
- Tools let the agent gather context and act on external systems
- An agent has three components :
  - The model, the tools and the instructions
- Models differ in strength, latency and cost, so mixing them is normal
- Agents need three kinds of tool :
  - Data tools, action tools and orchestration tools



- Intro AI Agents
- ▶ **LLM Fundamentals**
- LangChain Intro
- LangChain Components
- Agents and Tools
- LangGraph Intro
- Multi Agent Systems
- Model Context Protocol
- Running Agents

- What are LLMs?
- How Much of This Do We Need?
- Generative AI
- Neural Networks
- Transformer Architecture
- Encoder and Decoder
- Generative Pretrained Transformers
- Custom Built LLMs
- Word Embeddings
- Data Type Embeddings
- Embedding Approaches
- Embedding Dimensions
- Splitting in Tokens
- Creating Token Embeddings
- Next Word Generation
- Attention Mechanism
- Simple Self Attention

# What are LLMs?

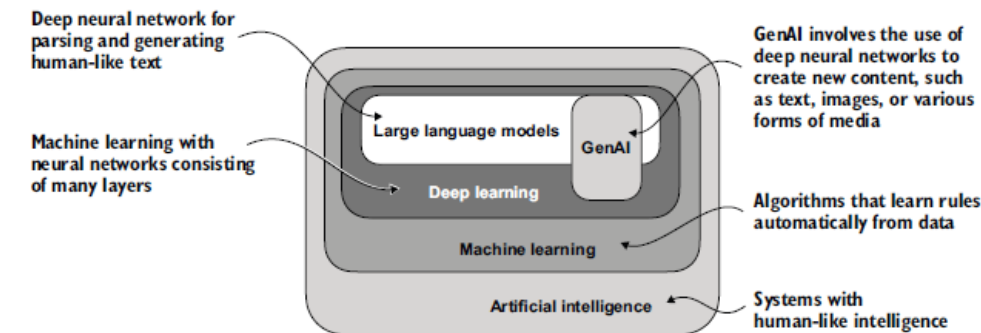
- A large language model predicts the next token, over and over
- That single mechanism is enough to answer questions, summarize, translate and write code
- Trained on a large slice of public text, then tuned to follow instructions :
  - Training reads text and adjusts weights ; nothing is looked up at run time
  - A model knows nothing that was not in its training data or its prompt
- Calling it autocomplete is fair but misleading :
  - Autocomplete finishes a word ; an LLM holds a plan across paragraphs
- The interesting part is not that it predicts, but what it had to learn to predict well :
  - Grammar, facts, tone, and enough reasoning to keep a long answer consistent

# How Much of This Do We Need?

- None of the mathematics in this module is needed to build an agent
- Every agent in this course is ordinary Python wrapped around an HTTP call
- The module is here for the vocabulary the rest of the course leans on :
  - Tokens explain why a long conversation costs more, and eventually breaks
  - Embeddings explain how a vector store finds text by meaning
  - The context window explains why an agent forgets
  - Attention explains why a model weighs the whole prompt rather than its tail
- Follow the ideas ; the arithmetic is background
- Nothing later in the course asks us to implement any of it

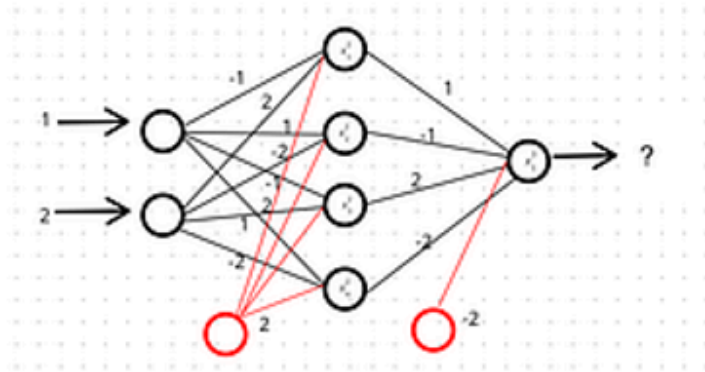
# Generative AI

- Generative AI is the umbrella term for models that produce new content
- Language models are one branch ; image, audio and video models are others
- What they share is the transformer architecture underneath
- The transformer's contribution is attention :
  - Every token can look at every other token when deciding what comes next
  - Before attention, models read strictly left to right and forgot the beginning

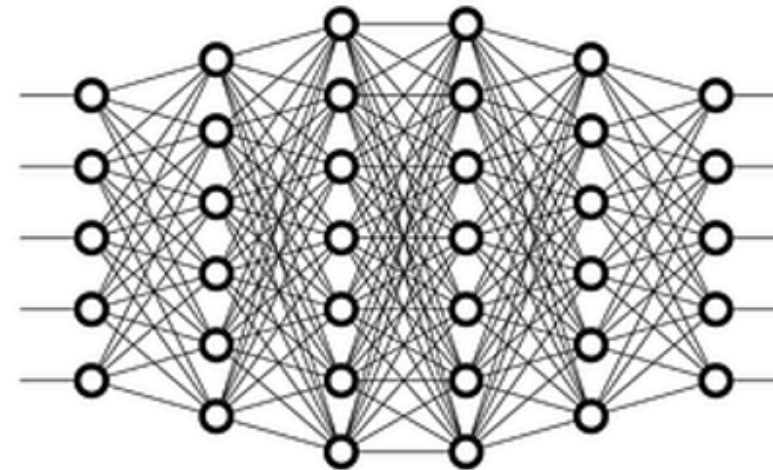


# Neural Networks

**Model capacity** refers to the ability of a machine learning model to capture and represent complex patterns in data. It is a measure of the model's potential ability to learn and remember relationships between inputs and outputs.



Shallow neural network - low-capacity model

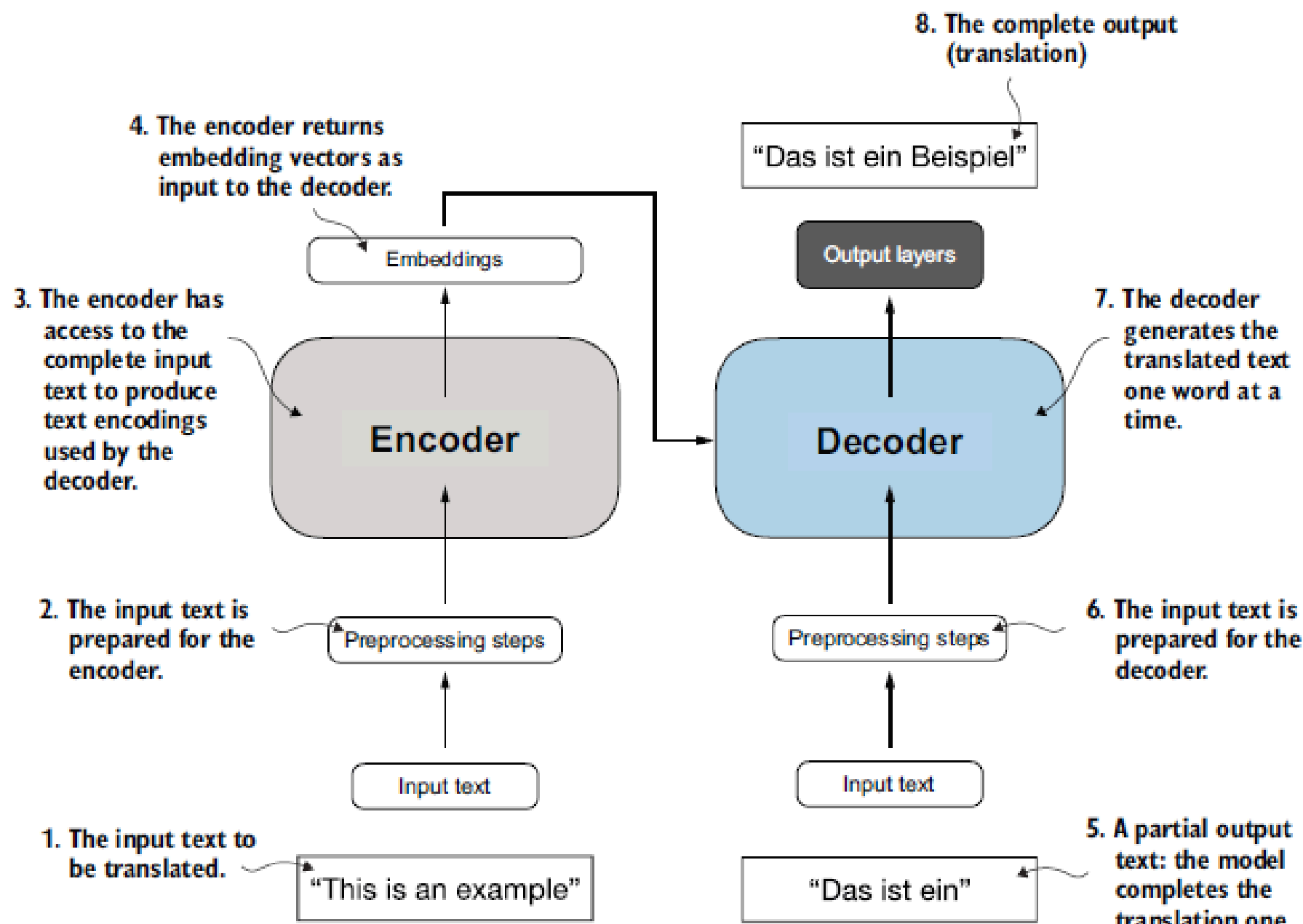


Deep neural network - a model with "larger" capacity 229 parameters for the example above 400 billion parameters for LLMs, for example

# Transformer Architecture

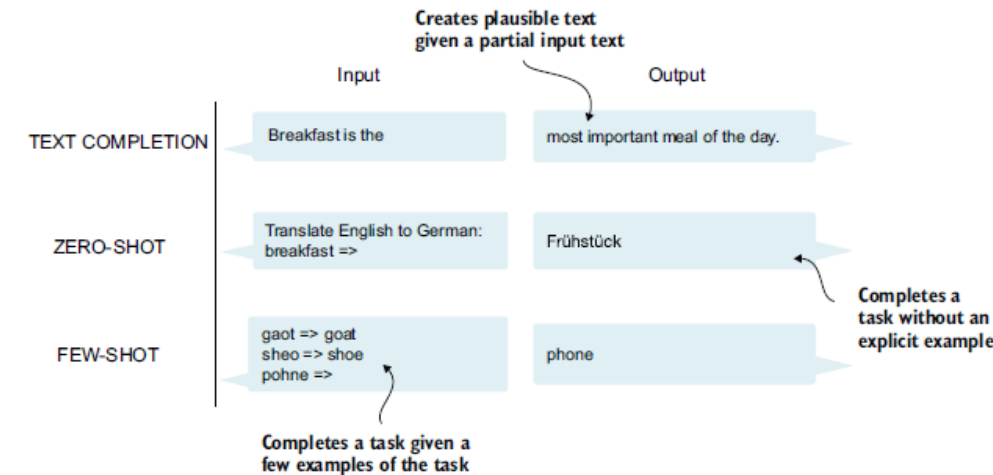
- Introduced in 2017 in "Attention Is All You Need", for machine translation
- Two submodules, and most models use only one of them :
  - The encoder turns input text into vectors carrying its context
  - The decoder turns vectors back into text, one token at a time
- BERT is encoder-only, and is used for classification and search
- GPT is decoder-only, and is used for generation :
  - Translation models used both, which is where the diagram comes from
- Each module is a stack of identical layers :
  - Every layer applies self-attention, then a small feed-forward network
- GPT-3 stacks 96 of them

# Encoder and Decoder



# Generative Pretrained Transformers

- **GPT is generative, pretrained, transformer :**
  - Pretrained means trained once on general text, before it ever sees our task
- **Generation is autoregressive : each new token is appended and fed back in**
- **A 500-token answer is therefore 500 forward passes :**
  - Output length costs more than input length, in both time and money
- **It also explains streaming :**
  - The tokens arrive one at a time, as each is generated





# Custom Built LLMs

- Almost nobody trains a model from scratch ; the cost runs to millions
- Building a small one anyway is the fastest way to understand the mechanics
- What teams do in practice, in increasing order of effort :
  - Prompt a hosted model
  - Retrieve context and put it in the prompt, which is RAG
  - Fine-tune an open model on domain data
  - Train from scratch, essentially never
- Reasons to run our own model rather than call an API :
  - Sensitive data that may not leave the building
  - Latency and cost at high volume
  - A guarantee the model will not change underneath us
- A fine-tuned 7B model often beats a general model on one narrow task

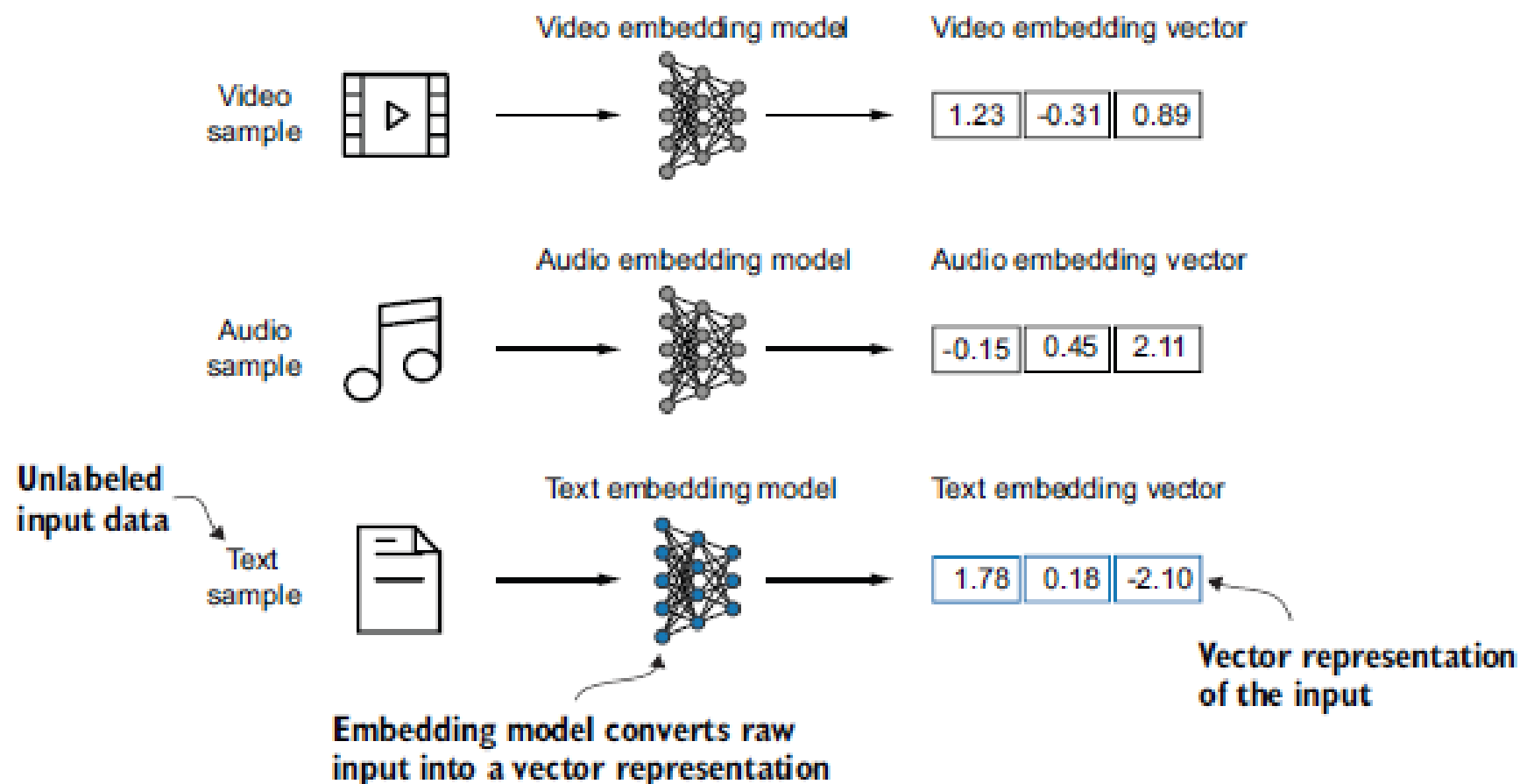
# Word Embeddings

- A network computes on numbers, so text must become numbers first
- An embedding maps a token to a vector of floats
- Vectors are arranged so that distance carries meaning :
  - Words used in similar contexts land near each other
  - Direction means something too, which is why arithmetic on word vectors works
- Every data type needs its own embedding model ; a text model cannot embed audio
- Whole sentences and paragraphs embed the same way :
  - Sentence embeddings make retrieval possible, and are the basis of RAG

```
from langchain_openai import OpenAIEmbeddings

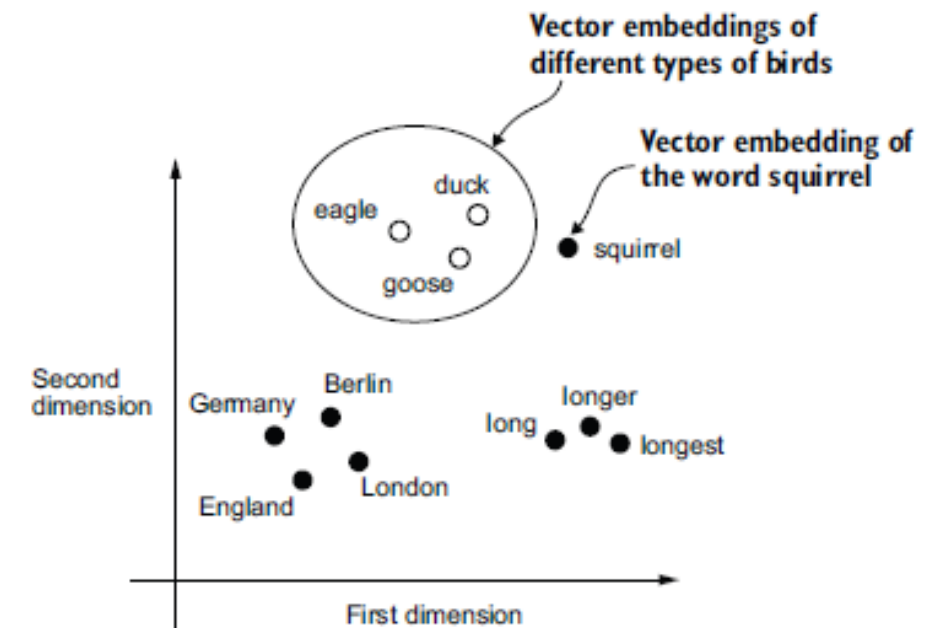
emb = OpenAIEmbeddings(model="text-embedding-3-small")
vector = emb.embed_query("a small striped cat")
print(len(vector), vector[:4]) # 1536 [0.021, -0.008, ...]
```

# Data Type Embeddings



# Embedding Approaches

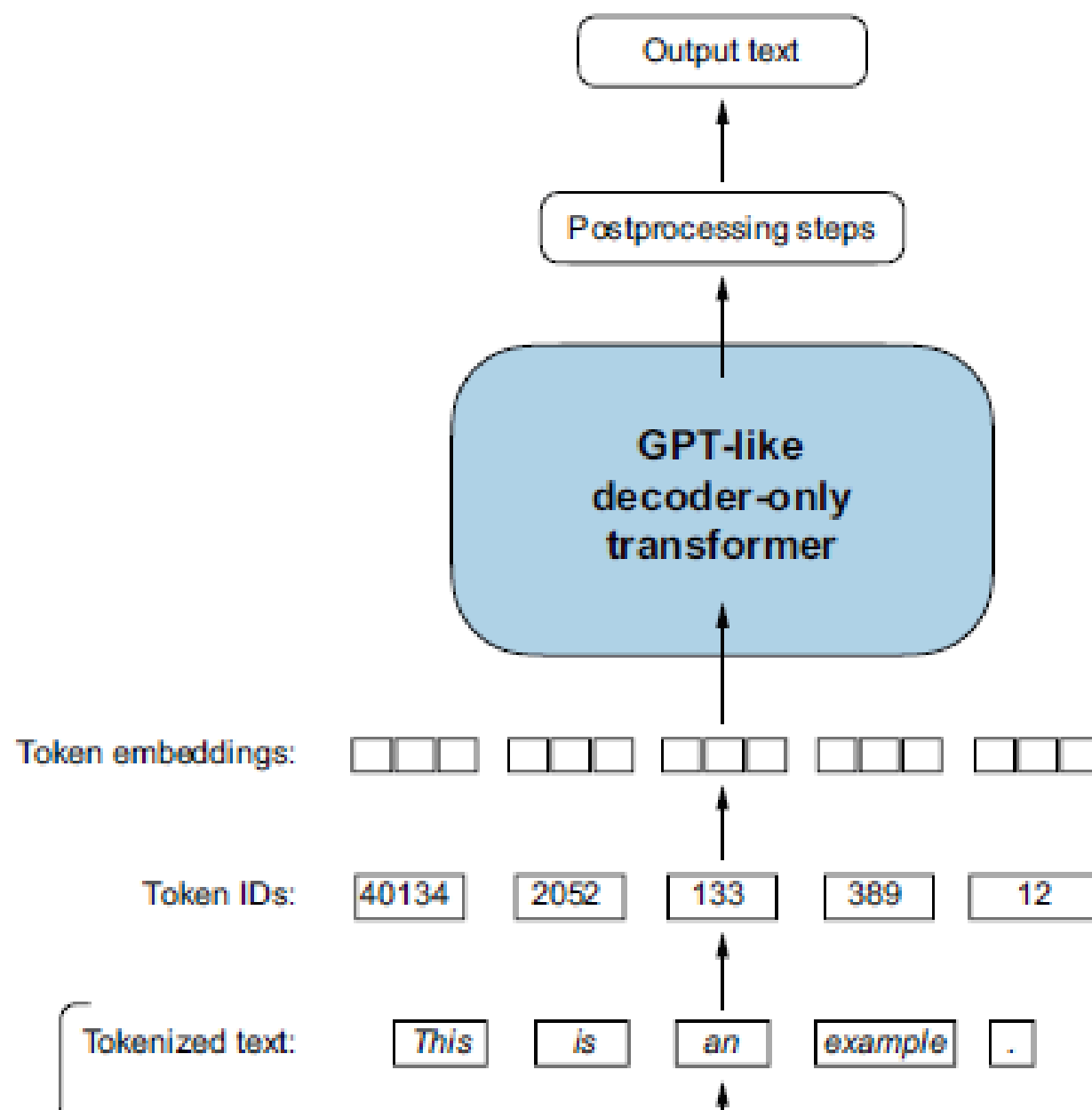
- Word2Vec, from 2013, made the idea concrete and is still the clearest example
- Its assumption : words appearing in similar contexts have similar meanings
- Trained by hiding a word and asking the network to guess it from its neighbours :
  - No labels are needed, which is why it scaled to enormous corpora
- Projected down to two dimensions, related terms visibly cluster
- One fixed vector per word is its limitation :
  - The word *bank* gets one vector, whether the sentence is about money or a river
  - Transformers fix this by computing the vector from the whole sentence



# Embedding Dimensions

- More dimensions hold finer distinctions, at a cost in compute and storage
- Typical sizes :
  - GPT-2 small, 117M parameters, uses 768
  - GPT-3, 175B parameters, uses 12,288
  - `text-embedding-3-small` returns 1536, and can be truncated to 256
- Beyond three dimensions nothing can be drawn :
  - Every embedding picture we have seen is a projection
- Bigger is not automatically better :
  - A 1536-dimension vector costs six times the storage of a 256-dimension one
  - On a small corpus the shorter vector often retrieves just as well

# Splitting in Tokens



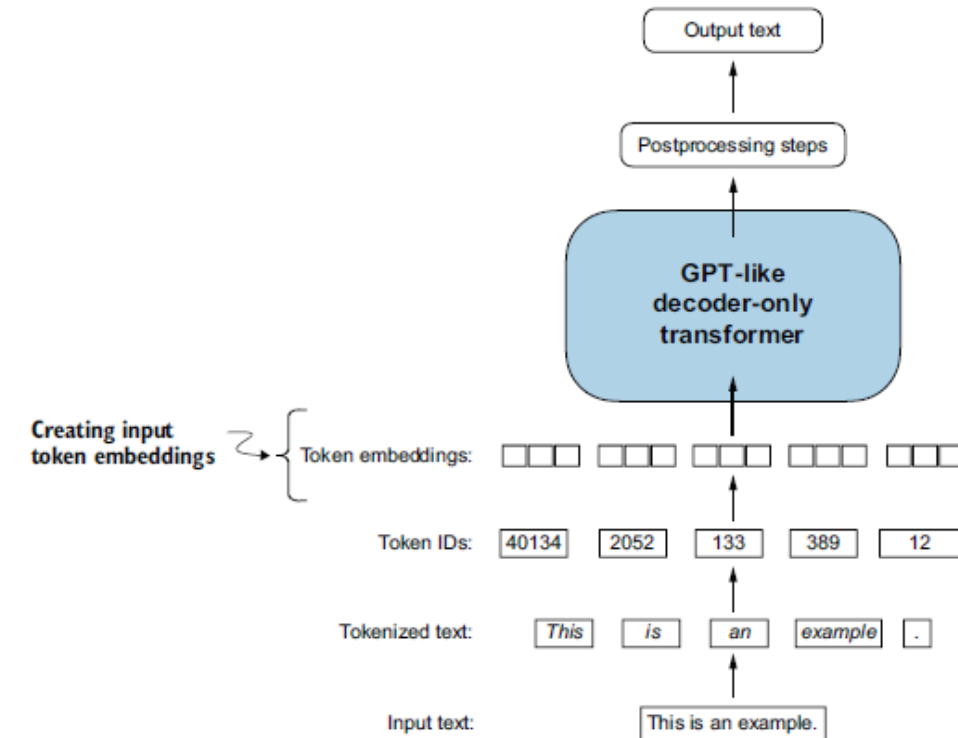
This section covers the

# Creating Token Embeddings

- **Tokens are not words :**
  - Common words are a single token, rare ones split into several pieces
- **A tokenizer maps each token to an integer ID, which indexes a lookup table**
- **That table starts as random numbers and is learned during training**
- **Counting tokens matters in practice :**
  - Context limits and billing are both measured in tokens

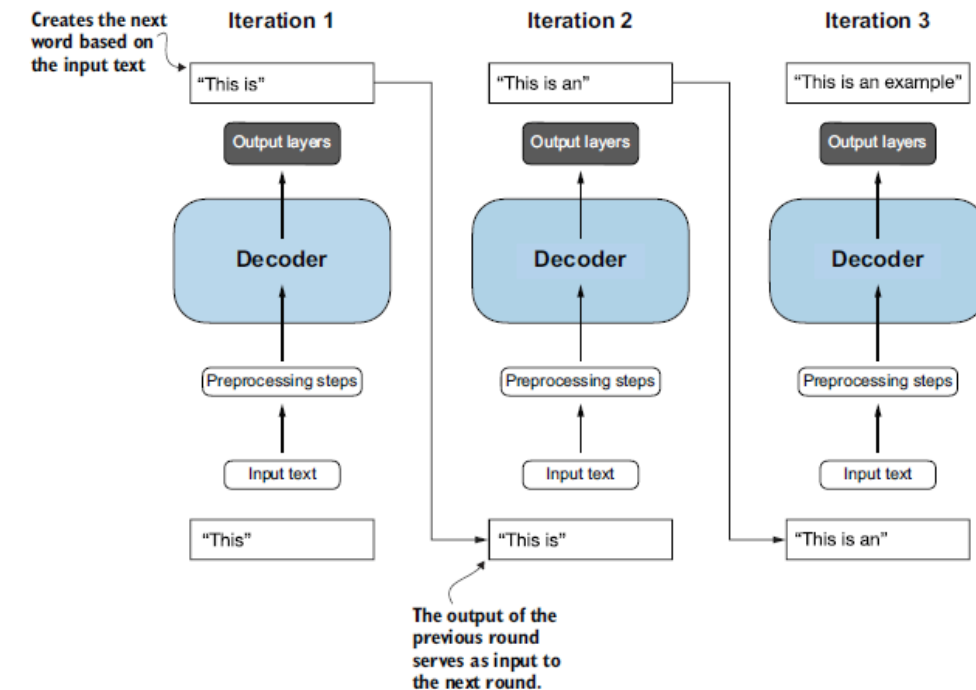
```
import tiktoken

enc = tiktoken.get_encoding("cl100k_base")
ids = enc.encode("Tokenisation is unavoidably fiddly.")
print(len(ids)) # 9
print([enc.decode([i]) for i in ids])
['Token', 'isation', ' is', ' unavoid', 'ably', ' f', 'idd', 'ly',
```



# Next Word Generation

- At each step the model scores every token in its vocabulary
- Sampling picks one, and temperature decides how adventurously :
  - `temperature=0` takes the most likely token and is near-deterministic
  - Higher values sample from the tail, which reads as more creative
- **GPT-3 was never trained to translate, yet it could :**
  - Capabilities appearing without being trained for are called emergent
  - They are also why evaluating these models is hard



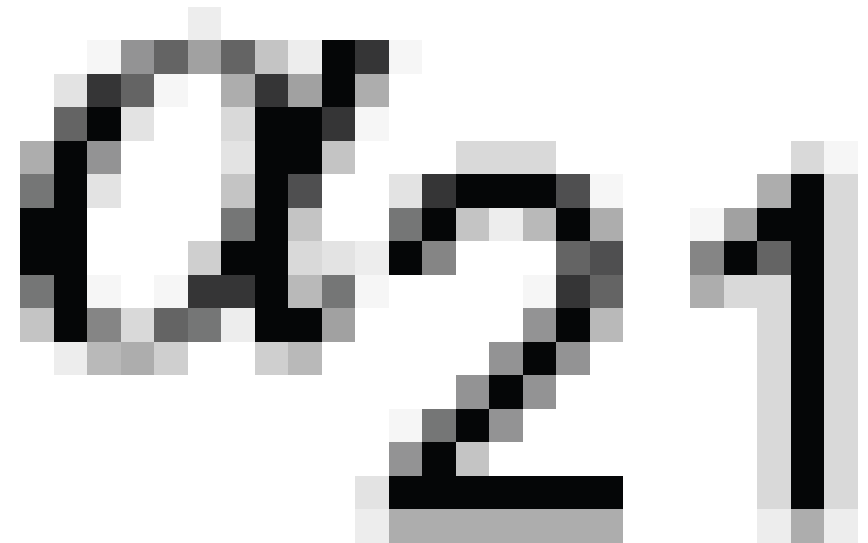


# Attention Mechanism

- Attention lets a token look at other tokens and weigh how much each one matters
- It is what resolves a pronoun :
  - *The cup did not fit in the box because it was too big ; what was too big?*
- Each token produces three vectors :
  - Query, what this token is looking for
  - Key, what this token offers to others
  - Value, the content it contributes if attended to
- A score is the dot product of one token's query with another's key
- Scores are normalised into weights that sum to one :
  - The output is then the weighted sum of the value vectors
- Self-attention means every token scores every other token in the same sequence
- Multi-head attention runs the same computation several times in parallel :
  - One head may track grammatical subject, another long-range reference
  - GPT-3 uses 96 heads per layer

# Simple Self Attention

- The goal is one context vector per token, mixing in information from the others
- Working through  $z(2)$ , the context vector for the second token :
  - Score token 2's query against every key, including its own
  - Normalise the scores so they sum to one ; these are the attention weights
  - Multiply each token's value by its weight and add them up
- A high weight means that token strongly shapes this position :
  - The weights are the part worth inspecting when a model misreads a sentence
- Every position is computed the same way, and all of them at once



# Summary : LLM Fundamentals

- An LLM predicts one token at a time ; everything else follows from that
- The transformer's contribution is attention, letting every token see every other
- GPT is decoder-only and autoregressive :
  - Output length drives latency and cost
- Text becomes numbers through tokenization, then embedding
- Tokens are not words, and both context limits and billing count tokens
- Embedding dimension trades nuance against compute and storage
- Attention scores a query against keys and returns a weighted sum of values :
  - Multi-head attention runs many such comparisons at once

- Intro AI Agents
- LLM Fundamentals
- ▶ **LangChain Intro**
- LangChain Components
- Agents and Tools
- LangGraph Intro
- Multi Agent Systems
- Model Context Protocol
- Running Agents

- What is LangChain?
- LangChain Components
- Benefits of LangChain
- Applications of LangChain
- LangChain Origins
- LangChain Ecosystem
- LangChain Installation
- Simple Prompt
- ChatModels
- Message List
- Other Models
- Other Messages
- LLM Use Case Scenarios
- Text Generation and Classification
- Sentiment and Document Analysis
- Translation, Q&A and Summarization

# What is LangChain?

- Open-source framework for building LLM applications and agents
- Available in Python and JavaScript
- Its value is a standard interface over things that all work differently :
  - Model providers, vector stores, document loaders, retrievers, tools
- Swap OpenAI for Anthropic or a local Ollama model by changing one line :
  - Retries, fallbacks, streaming and batching come with the interface
- Components compose with the pipe operator, through LCEL
- The framework is thin glue between our program and the providers

# LangChain Components



# Benefits of LangChain

- **The parts we assemble applications from :**
  - Models, the wrapper over any chat or embedding provider
  - Prompt templates, which define how input is formatted
  - Chains, which wire components so one output feeds the next
  - Memory, which carries conversation history between calls
  - Vector stores, which hold embeddings for retrieval
  - Agents, which let the model decide which tools to call
- **Each has one interface regardless of the provider behind it**
- **That uniformity is the actual benefit :**
  - A retriever swap does not ripple through the rest of the application

# Applications of LangChain

- Chatbots and virtual assistants that remember earlier turns and call APIs
- Document question answering :
  - Query PDFs, contracts or research papers instead of searching them by hand
- Knowledge management across structured and unstructured company data
- Workflow automation :
  - Ticket resolution, report generation, CRM updates
- Natural language to SQL for business intelligence
- The common shape is the same in all of them :
  - Retrieve something, put it in a prompt, act on what comes back



# LangChain Origins

- Created in 2022, when GPT-3 made LLM applications suddenly practical
- Every project was reinventing the same four things :
  - Chaining prompts into a workflow
  - Keeping track of conversation history
  - Querying databases and search engines
  - Wiring external tools around the model
- LangChain packaged them so applications could be assembled rather than glued
- The tradeoff is the usual one for a framework :
  - Prototypes come together in an afternoon
  - Debugging means understanding somebody else's abstraction
- Ten lines is enough to reach OpenAI, Anthropic, Google or a local model

# LangChain Ecosystem

- **LangChain is rarely used alone :**
  - Normally paired with LangSmith for observability and LangGraph for control flow
- **LangSmith is mission control for an LLM application :**
  - Traces runs, inspects prompts and outputs, compares versions
  - Answers what was sent, what came back, what each call cost
  - Shows how behaviour changed after a refactor
- **LangGraph describes a workflow as a state graph :**
  - Nodes are actions, tools or model calls
  - Edges are transitions that depend on the current state
- **Graph form suits agents navigating a dynamic workflow**

# LangChain Installation

- Install the framework and one provider package

```
pip install langchain langchain-openai python-dotenv
```

- Other providers plug in the same way :
  - `langchain-anthropic`, `langchain-google-genai`, and so on
  - Each has its own chat class ; agent code is identical once the model exists
- Any OpenAI-compatible endpoint works through `langchain-openai` :
  - Set `base_url` and point it at Ollama, vLLM or a hosted gateway
- Model names in this course are a September 2026 snapshot :
  - They date faster than anything else here ; check the provider's model list

# Simple Prompt

- Every provider follows the same three steps :
  - Load the key, build the chat model, invoke it

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv() # read .env into the environment
api_key = os.getenv("OPENAI_API_KEY")
if not api_key:
 raise ValueError("Missing API key in file .env!")

llm = ChatOpenAI(model="gpt-5.6-luna", temperature=0, api_key=api_key)
response = llm.invoke("Write a short greeting in English.")
print(response.content)
```

- Only the import and the model class change per provider :

```
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model="claude-sonnet-5", temperature=0, api_key=api_key)

from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini-3.7-flash", temperature=0, api_key=api_key)
```

# ChatModels

- **ChatModel is LangChain's wrapper around a model :**
  - Works in terms of messages rather than plain text
- **Start conversation by handing model two messages with a role :**

SystemMessage and HumanMessage

- **As a result of the conversation the model returns and AIMessage**
- **SystemMessage sets behavior and context for the model :**
  - **SystemMessage** is the instruction layer and not part of the dialogue
  - Shapes how model responds to everything else e.g. respond in "Dutch"
- **HumanMessage is input from user, questions and prompts :**
  - When passing plain string to `.invoke()`, this is what LangChain wraps it into
- **AIMessage is model's response also called assistant message :**
- **`.invoke()` returns an AIMessage ; the text is on `.content`**
- **In a multi-turn conversation :**
  - Accumulate history by appending each **AIMessage** and next **HumanMessage** to list

# Message List

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")

llm = ChatOpenAI(model="gpt-5.6-luna", temperature=0, api_key=api_key)

system_message = SystemMessage(content="You are a helpful assistant. Always answer in Dutch.")
human_message = HumanMessage(content="Write a short greeting.")

messages = [system_message, human_message]

response = llm.invoke(messages)

print("Model response:\n")
print(response.content)

print("\nType of SystemMessage:", type(system_message))
print("Type of HumanMessage: ", type(human_message))
print("Type of response: ", type(response))
```

Model response:

# Other Models

- **ChatModels are what is used 95% of the time :**
  - Were introduced by OpenAI in 2023 and other providers followed
- **Previously models took in strings and returned a string :**
  - Older LLM type (BaseLLM) was text-in, text-out interface, no concept of roles
- **There are model types for different jobs entirely :**
  - Most models are often served through chat interface, then ChatModels are used
- **Embedding models like OpenAIEmbeddings and OllamaEmbeddings :**
  - Turn text into vectors, backbone of retrieval/RAG and semantic search
  - They expose `embed_query` and `embed_documents` rather than `invoke`
- **Rerankers / cross-encoders :**
  - Used to reorder retrieved documents by relevance
- **Multimodal capability increasingly lives inside ChatModels :**
  - Modern ones accept images and sometimes audio as message content
  - Rather than being a separate class

# Other Messages

- **Growing list of message is the conversation memory :**

- Model is stateless, so we resend the relevant history each turn

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

messages = [
 SystemMessage(content="You are a terse assistant."),
 HumanMessage(content="What's the capital of France?"),
 AIMessage(content="Paris."),
 HumanMessage(content="And of Spain?"), # model now has the full context
]
```

- **ToolMessage** carries result of tool call back to the model :

- Central to agents is that Model emits an **AIMessage** containing tool calls

- **Code runs the tool and output is fed back as a ToolMessage :**

- Model then continues with that result in hand

- **FunctionMessage** is deprecated predecessor to **ToolMessage** :

- When API called this "functions" rather than "tools"

- **RemoveMessage** is a LangGraph-specific one :

- Doesn't represent dialogue but instructs graph to delete message from accumulated state



# LLM Use Case Scenarios

- **LLMs apply wherever text is analysed or generated :**
  - Same chat model serves every scenario, only the prompt changes
  - Model is built once and reused in the examples that follow

```
llm = ChatOpenAI(model="gpt-5.6-luna", temperature=0, api_key=api_key)
```

- **Seven scenarios are covered here :**
  - Text generation, classification, sentiment analysis
  - Document analysis, translation, question answering, summarization

# Text Generation and Classification

- **Text Generation :**

- Coherent, context-aware text from a short prompt
- Drafting emails, blog posts, marketing copy or code comments

```
response = llm.invoke("Generate a recipe for sweet cheesecake with tuna, broccoli and onion.")
```

- **Classification :**

- Assigns categories without a hand-built ML pipeline
- Routing support tickets, tagging by topic, detecting spam

```
articles = ["The government announced new tax reforms today.",
 "The local team won the championship in a thrilling match.",
 "New advancements in AI are reshaping the tech industry."]
for subject in articles:
 response = llm.invoke("Classify into Politics, Sport, Technology, Culture, Health. "
 f"Return a single word. Text: {subject}")
```

# Sentiment and Document Analysis

- **Sentiment Analysis :**

- Reads nuance : sarcasm, mixed feelings, polite but negative feedback
- Brand monitoring, product reviews, detecting customer frustration

```
reviews = ["This product is amazing! I loved it.",
 "I am very disappointed. The product broke after one use.",
 "It's okay, does the job but nothing special."]
for review in reviews:
 response = llm.invoke("Rate the sentiment of this review. "
 f"Return one word, positive, negative or neutral: {review}")
 print(review, response.content)
```

- **Document Analysis :**

- Interprets contracts, financial reports and manuals

```
document = open("../data/annual_report.html").read()
response = llm.invoke("Find the company's annual revenue in the document below.\n"
 f"{document}")
```

# Translation, Q&A and Summarization

- **Machine Translation :**

- Aware of idiom, domain terminology and cultural context

```
response = llm.invoke("Translate into Dutch:\n"
 "I'm a foreigner and I don't speak German fluently.")
```

- **Question Answering :**

- Answers are grounded in the text supplied with the question

```
response = llm.invoke("Answer using only the text below.\n"
 "Chains in LangChain are data flows between prompts, models and parsers.\n"
 "Question: What are Chains used for?")
```

- **Summarization :**

- Long input is truncated to stay inside the context window

```
document = open("../data/nad_niemnem.txt", encoding="utf-8").read()
response = llm.invoke(f"Write a summary of about 500 words.\n{document[:1800]}")
```

# Summary : LangChain Intro

- LangChain is a standard interface over models, stores, loaders and tools
- Its point is that swapping a provider does not ripple through the application
- Components compose with the pipe operator, through LCEL
- It is rarely used alone :
  - LangSmith for observability, LangGraph for control flow
- A `ChatModel` works in messages
- `SystemMessage` instructs, `HumanMessage` asks, `AIMessage` answers
- The model is stateless, so history is resent every turn
- One chat model covers text generation, classification, sentiment, translation, question answering and summarization ; only the prompt changes

- Intro AI Agents
- LLM Fundamentals
- LangChain Intro
- ▶ **LangChain Components**
- Agents and Tools
- LangGraph Intro
- Multi Agent Systems
- Model Context Protocol
- Running Agents

- LangChain Packages
- Models
- Model Parameters
- Model Parameters
- PromptTemplates
- Using PromptTemplates
- Output Parsers
- Common Output Parsers
- Chains
- Context Window
- Context Engineering
- Controlling the Context Window
- Memory in LangChain
- Memory Types
- Memory in Chains
- Memory as Summary
- Context Failure Modes
- Vector Databases
- Calculations with Text
- Embeddings
- In Memory Store
- RAG
- RAG Pipeline
- RAG Operation
- Simple RAG Chain
- Tool Results and the Context Window

# LangChain Packages

- **The package layout can be confusing, and the history explains why :**
  - The library has been split and resplit several times as it grew
- **langchain-core is the foundation :**
  - It holds the essential abstractions and interfaces
  - Prompts, parsers, runnables, chains and the model interfaces
- **langchain-community is a large collection of integrations :**
  - Connectors to tools, vector databases, memory backends and external systems
  - Being sunset since 2026 ; each integration moves to its own package
- **langchain\_classic is the compatibility layer :**
  - It exposes the older, high-level APIs known from earlier versions
  - Classic `LLMChain`, the older agent interfaces and assorted utilities
- **On top of that, most providers ship a dedicated package :**

```
langchain-openai
langchain-anthropic
langchain-google-genai
```

# Models

- The central design fact is that a chat model is an abstraction :
  - The interface is `BaseChatModel` ; messages in, messages out
- LangChain is model-agnostic and integrates many different models :
  - OpenAI's GPT, Anthropic's Claude, Google's Gemini, Hugging Face models
- That flexibility lets us pick the best model for the use case
- Almost every model used in practice is a chat model :

`ChatOpenAI`, `ChatAnthropic`, `ChatOllama`

- There are two ways to instantiate one :
  - The provider-specific class, `ChatOpenAI(model="gpt-5.6-luna")` , used so far
  - The provider-agnostic factory, `init_chat_model("openai:gpt-5.6-luna")`
- The factory can also change model at run time :
  - Pass `config={"configurable": {"model": ...}}` on the call
  - The same code then moves between OpenAI and Anthropic without an edit
- Every chat model has the same `invoke` , `stream` and `batch` methods



# Model Parameters

- Model parameters are the settings passed to the model to control how it behaves
- LangChain splits them into two groups :
  - A handful of standard parameters that mean the same thing at every provider
  - Provider-specific parameters, passed as extra keyword arguments
- The standard set is short, and every provider honours it
- Temperature is the one that matters most in this course :
  - A low temperature makes the model deterministic and consistent
  - Tool-calling traces then reproduce from run to run

| Parameter                | Controls                              | Typical use                                                           |
|--------------------------|---------------------------------------|-----------------------------------------------------------------------|
| <code>temperature</code> | Randomness of the output              | <code>0</code> for deterministic/factual, higher for creative variety |
| <code>max_tokens</code>  | Maximum length of the <i>response</i> | Cap cost and prevent runaway output                                   |
| <code>timeout</code>     | Seconds to wait before giving up      | Stop a hung request                                                   |
| <code>max_retries</code> | Auto-retries on transient failure     | Robustness on flaky networks (default <code>6</code> )                |

# Model Parameters

- **max\_tokens** caps the length of the response :
  - It is not the context window ; it limits only what the model generates
  - Use it to control cost and to keep an answer from running long
- **max\_retries** limits how often a failed request is retried :
  - Chat models already retry up to six times by default
  - It is rarely worth setting, except to raise it on an unreliable connection
- **timeout** is the number of seconds a request may take before it fails :
  - The parameter reached for most often
- Anything beyond that set is provider-specific, and is passed as ordinary kwargs
- The common OpenAI parameters :
  - **top\_p** ; nucleus sampling, an alternative to temperature
  - **frequency\_penalty** and **presence\_penalty** discourage repetition
  - **seed** ; ask for reproducible sampling on a fixed prompt

# PromptTemplates

- **Plain strings can always be passed straight to the model :**
  - That gets messy as soon as the application grows past a handful of prompts
- **A prompt template defines the structure once and takes values at runtime :**
  - Placeholders such as `{question}` and `{language}` are filled in on each call
- **Templates also decide how inputs and outputs are formatted for the model**
- **They are where prompt engineering lives, which makes behaviour easier to control**
- **Two consequences follow :**
  - The instructions are easier to read and to maintain
  - Variants of the same prompt can be tested quickly against each other

```
prompt = PromptTemplate.from_template(
 "Suggest {number} names for a {domain} startup."
)
llm = ChatOpenAI(
 model="gpt-5.6-luna", # or "gpt-5.6-terra", "gpt-5.6-sol"
 temperature=0.7,
)
```

# Using PromptTemplates

- A reusable chat template comes from `ChatPromptTemplate.from_messages(...)` :

```
prompt = ChatPromptTemplate.from_messages([
 ("system", "You are an expert in {domain}. Answer concisely."),
 ("human", "{question}"),
])
```

- `format_messages` fills the placeholders and returns a list of messages :

```
messages = prompt.format_messages(
 domain="Python programming",
 question="What is a generator?"
)
llm = ChatOpenAI(model="gpt-5.6-luna", temperature=0)

response = llm.invoke(messages)
print(response.content)
```

A generator in Python is a special type of iterable that allows you to iterate over a sequence of values without storing them all in memory at once. It is defined using a function with the `'yield'` statement, which produces a value and pauses the function's state, allowing it to resume from that point on subsequent calls. This makes generators memory-efficient and suitable for handling large datasets or streams of data.

# Output Parsers

- Output parsers turn loose text into structured data
- Models return plain text, while applications usually need something typed
- A parser takes the raw response and does three things :
  - Enforces an expected format, such as JSON, a list or an enum
  - Turns it into a Python object or a Pydantic model
  - Fails loudly when the response does not match the schema
- Without one we end up writing ad-hoc string splitting and regexes :
  - The parser puts that formatting and parsing logic in a single place

```
prompt = ChatPromptTemplate.from_messages([
 ("system", "You are a helpful assistant. Keep your answers concise."),
 ("user", "Summarize in 1 sentence: {text}"),
])
parser = StrOutputParser()
Here the result would be identical without the parser
The value shows up once the parser sits inside a chain
messages = prompt.format_messages(text="LangChain builds LLM applications.")
response = llm.invoke(messages)
result = parser.parse(response.content)
```

# Common Output Parsers

- `StrOutputParser` returns the message content as a plain string
- `JsonOutputParser` parses into a dict or list, and can stream partial JSON
- `PydanticOutputParser` validates into a Pydantic model and returns a typed object
- `XMLOutputParser` parses XML-formatted output into a nested dict
- `CommaSeparatedListOutputParser` splits a comma-separated string into a list
- `NumberedListOutputParser` reads a numbered list into a Python list
- `MarkdownListOutputParser` reads a markdown bullet list into a Python list
- **Pick the parser that matches the format the prompt asked for :**
  - No parser rescues a prompt that never requested that format
  - The prompt and the parser have to be written together

# Chains

- A chain is a sequence of components wired so one output becomes the next input
- Chains exist because an LLM application is really a small pipeline :
  - Take the user input, format it into a prompt, send it, parse the result
- A step can query a model, reshape data, or call an external tool
- The standard way to build one is LCEL, the LangChain Expression Language :
  - Components are composed with the pipe operator
  - Every component is a Runnable, and a chain is itself a Runnable

```
prompt = ChatPromptTemplate.from_messages([
 ("system", "You are a concise assistant."),
 ("human", "{question}"),
])
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"question": "What is LCEL?"})
```

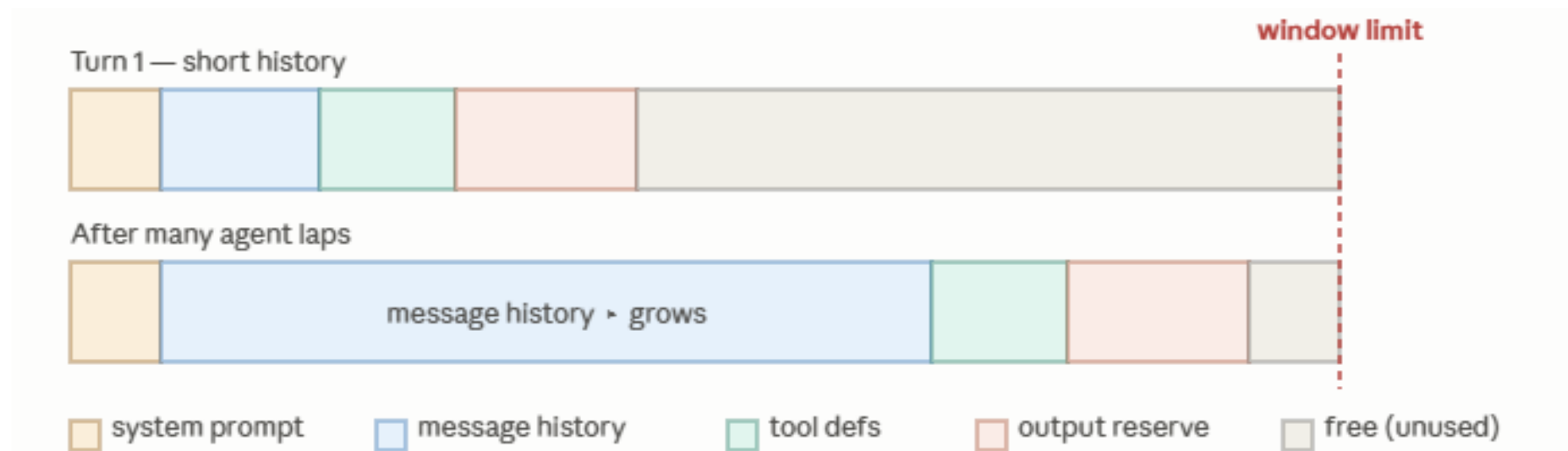
# Summary : LangChain Components

- LangChain is model-agnostic and integrates with many different models
- Model parameters control how the model behaves
- Prompt templates define the structure once and take values at runtime
- Output parsers turn loose text into structured data
- Chains wire components so one output becomes the next input :
  - LCEL composes them with the pipe operator, and every step is a Runnable



# Context Window

- The context window is the most text a model can consider in a single call :
  - The system prompt, the entire message history, the tool definitions, the answer
- The model is stateless, so the context window is everything it knows
- Nothing carries over between calls on its own
- Every `.invoke()` resends the whole message list :
  - The context window is what the model sees this time
- A long-running agent's history grows inside that fixed budget



# Context Engineering

- Prompt engineering is about the wording of one instruction
- Context engineering is about what occupies the window on every call :
  - The system prompt, the tool definitions, the history, the retrieved text
- The distinction matters because an agent rebuilds its prompt on every pass :
  - A chatbot has one prompt, written once and edited by hand
  - An agent assembles its prompt again at each turn, and it grows as it works
- The model carries nothing between calls, so everything it knows is resent
- The window is a fixed budget and the loop spends it :
  - A long run leaves less room for the instructions that started it
- Four strategies control that budget, and the rest of this module is those four

# Controlling the Context Window

- **Write : keep it outside the window and hold a reference to it**
  - A checkpoint, a file, a database row
  - The agent fetches it only when the task needs it
- **Select : bring back only what this turn requires**
  - Retrieval over a vector store ; filtering the tool list
- **Compress : keep the substance and drop the tokens**
  - Summarising older turns ; trimming a tool result down to its answer
- **Isolate : give a sub-task its own window**
  - A sub-agent's failed attempts never reach the caller's history
- **Every technique in this module is one of the four :**
  - Memory is write and compress, retrieval is select, sub-agents are isolate
- **Naming the missing one is usually enough to explain a slow or expensive agent**

# Memory in LangChain

- Everything is stateless by default ; each `invoke` is independent
- Memory is what carries the conversation history forward into the next call
- Short-term memory spans a single chat session
- Long-term memory spans separate conversations :
  - Recalling a fact about a user days later, in a new session
  - Stored in a database or a vector store, and retrieved on demand
- History holds the whole conversation and is injected message by message

```
history = InMemoryChatMessageHistory()
history.add_user_message("Buenos dias!")
history.add_ai_message("Hello!")
history.add_user_message("What's your name?")
history.messages
```

# Memory Types

- **Carrying previous messages forward runs into two constraints :**
  - The cost of the tokens, and the size of the context window
- **Full history keeps every message and attaches them all on each call :**
  - As the dialogue grows, the prompt grows with it
  - Cost rises, and the context window limit is eventually hit
- **Three pieces implement it :**
  - `RunnableWithMessageHistory` wraps the chain or the runnable
  - `InMemoryChatMessageHistory` stores the messages
  - A `MessagesPlaceholder("history")` marks where they land in the prompt
- **Each session is identified by an ID, such as a user ID or a session token :**
  - The wrapper stores the incoming and outgoing messages against that ID
  - On the next call the right history is injected into the placeholder
- **Summary memory stores a compressed summary instead of the full conversation**

# Memory in Chains

- Wrap an LCEL chain with `RunnableWithMessageHistory` :
  - A function maps a session ID to that session's message store
  - The wrapper loads the past messages before each call and appends after it

```
store = {}
def get_session_history(session_id: str):
 if session_id not in store:
 store[session_id] = InMemoryChatMessageHistory()
 return store[session_id]

chain_with_memory = RunnableWithMessageHistory(
 chain, get_session_history,
 input_messages_key="question",
 history_messages_key="history")
```

- The prompt needs a `MessagesPlaceholder` for that history to land in

```
prompt = ChatPromptTemplate.from_messages([
 ("system", "You are a helpful assistant."),
 MessagesPlaceholder(variable_name="history"),
 ("human", "{question}")])
```

# Memory as Summary

- **Summary memory exists because conversation history only grows :**
  - A long conversation eventually overflows the context window
  - Keeping every message verbatim costs more tokens on every single call
- **It periodically compresses the older turns :**
  - One paragraph summarising twenty past messages
  - Plus the few most recent messages, kept in full
- **Full history is the most accurate option and grows without bound**
- **Window memory keeps the last N turns, bounding size but forgetting everything older**
- **Summary memory sits between the two :**
  - It bounds the size and keeps the substance of the old turns
  - It costs some detail, and an extra model call to produce the summary

```
--- Show the resulting summary and what stays in full ---
print(f">>> UPDATED SUMMARY: {summary}")
print(f">>> MESSAGES STILL KEPT IN FULL: {len(recent)}")
```

# Context Failure Modes

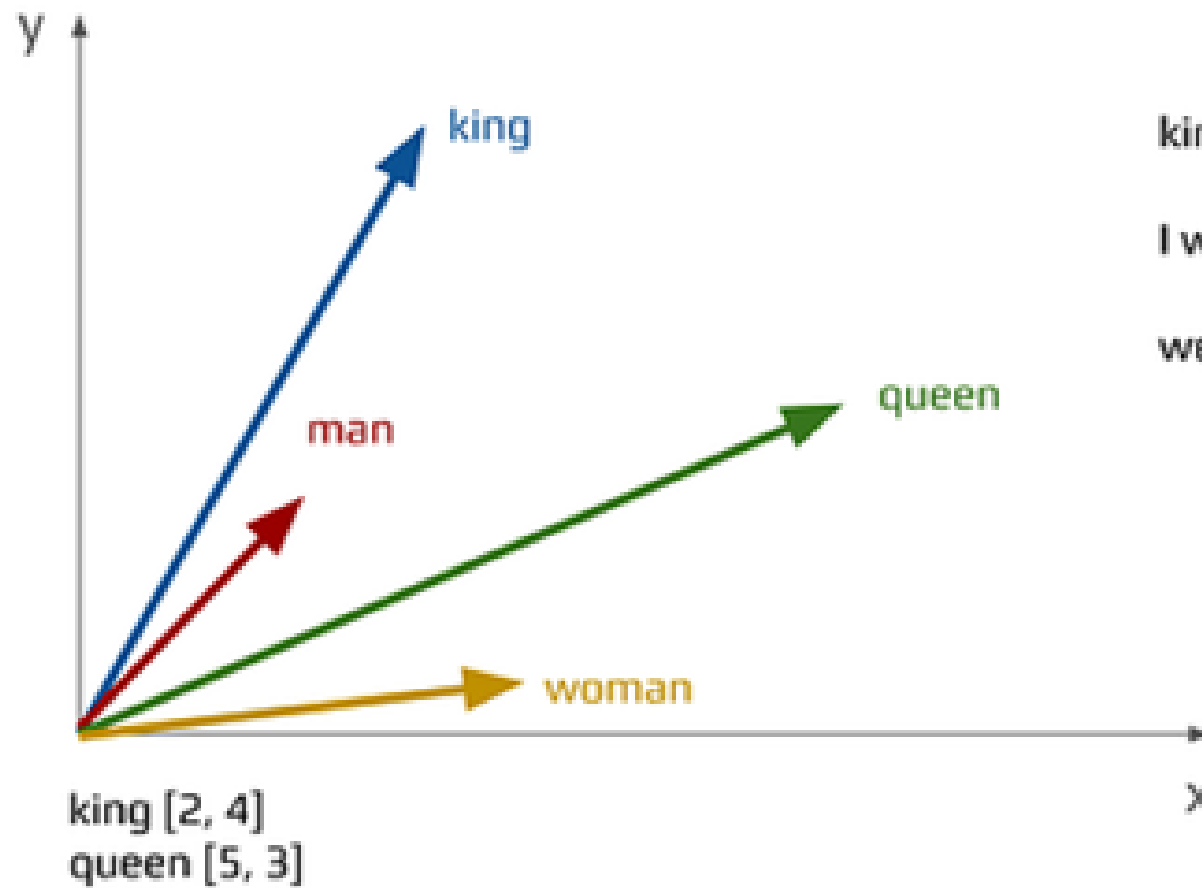
- **Distraction :**
  - Irrelevant history stays in the window and steers the model off the task
- **Rot :**
  - Recall degrades toward the middle of a long window
  - More context is therefore not reliably better than less
- **Poisoning :**
  - A wrong fact written to long-term memory is re-injected in every session
  - It survives a restart, and the transcript does not show where it came from
- **Clash :**
  - Two retrieved passages contradict each other, and the model picks one
- **All four produce a confident answer rather than an error :**
  - A test that only checks the run completed will not see any of them



# Vector Databases

- **A normal database finds things by exact match :**
  - Ask for the row where name is "Einstein" and it either matches or it does not
- **A vector database stores text as embeddings :**
  - Lists of numbers that capture meaning rather than exact wording
- **An embedding model turns text into a point in a high-dimensional space :**
  - Similar meanings land close together, unrelated texts land far apart
- **The query is embedded the same way as the stored text :**
  - The store returns the entries nearest the question ; a similarity search
- **Because meaning is numeric, text can be compared and combined arithmetically**
- **Similarity search is the foundation of RAG, retrieval-augmented generation :**
  - Retrieve the relevant chunks and feed them to the model as context
  - The model answers using information it was never trained on

# Calculations with Text



king - man + woman  $\approx$  queen

I walked - I walk  $\approx$  I ran - I run

we swim - we  $\approx$  I swim - I

# Embeddings

- Embeddings here are created with `SentenceTransformer` :

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
model.max_seq_length = 256

sentences = [
 "dinosaurs live in africa but in different time dimension",
 "this is sentence about little cat that liked this eat fast food",
 "this is the another sample sentence which is here just this not"
]
embeddings = model.encode(sentences, normalize_embeddings=True)
```

embeddings

```
array([[-0.04987684, 0.03634832, 0.01747593, ..., -0.05154557,
 0.01327896, -0.05160031],
 [0.07967843, 0.06043371, 0.0579344 , ..., 0.14950444,
 0.06014551, 0.02409914],
 [-0.008951 , 0.05474586, 0.08349688, ..., 0.00953911,
 0.09756645, 0.00701754]], dtype=float32)
```

```
print(len(embeddings[1]))
```

384

# In Memory Store

- LangChain integrates with vector databases to store and search those embeddings
- Similarity search is what they are for :
  - The query is converted into a vector
  - That vector is compared against the vectors already stored
  - The nearest entries come back as the relevant documents
- Vector stores carry document retrieval, knowledge base lookup and context-based search
- `InMemoryVectorStore` keeps the index in the process, which is enough to learn on

```
1. Some documents to act as our knowledge base
docs = [
 Document(page_content="SpiralTrain is training company based in Houten, Netherlands."),
 Document(page_content="The LangChain course covers chains, agents, tools, and memory."),
 Document(page_content="SpiralTrain courses can be delivered on-site or online."),
 Document(page_content="The Python course lasts 3 days and covers the language fundamentals."),
]

2. Embed the documents and store them in a vector store
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = InMemoryVectorStore.from_documents(docs, embedding=embeddings)
```

# RAG

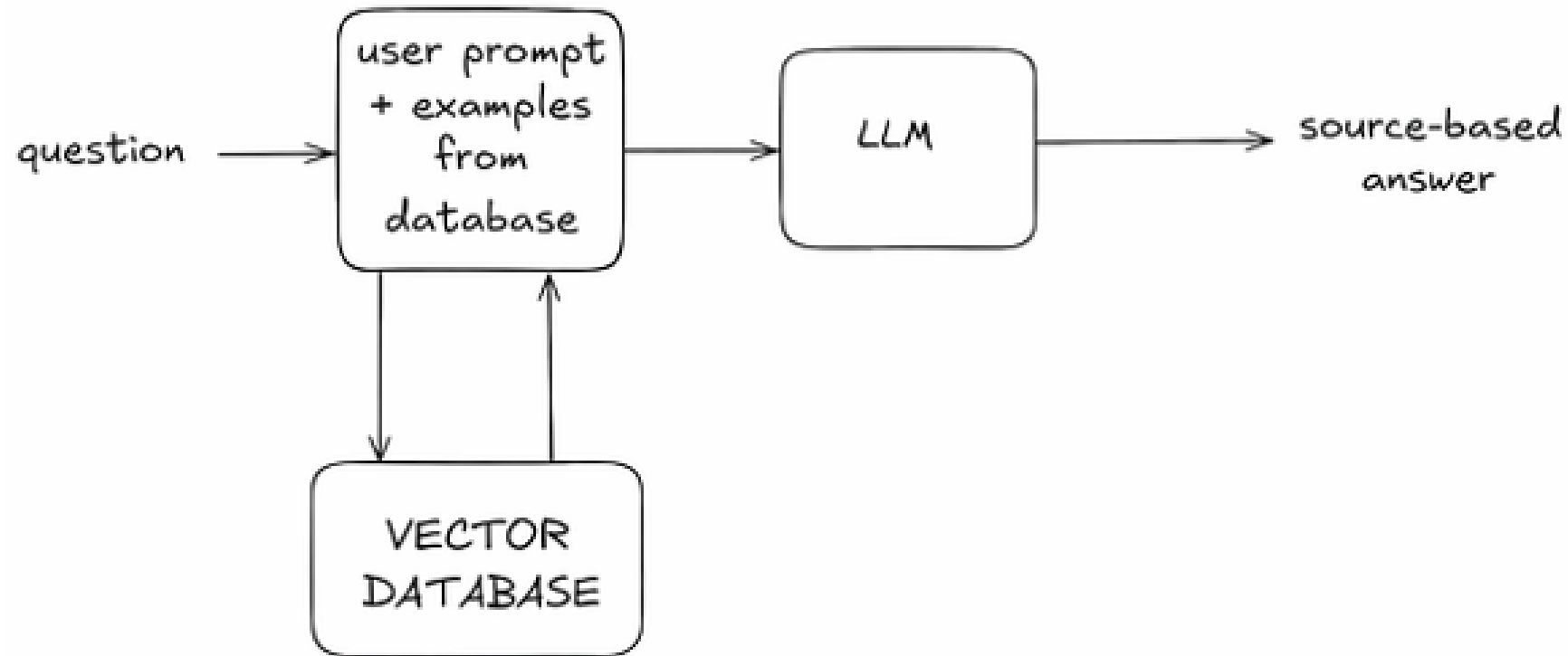
- **Retrieval-Augmented Generation connects a model to external data :**
  - Databases, documents or the web
  - It addresses the model's biggest weakness, its fixed knowledge
- **Four stages :**
  - Query ; the user asks a question
  - Retrieval ; the system searches a knowledge base
  - Augmentation ; the retrieved text is added to the prompt
  - Generation ; the model answers from its own knowledge plus that context
- **Hallucination drops, and answers can carry citations**
- **The usual applications are question answering and document analysis**

# RAG Pipeline

- **The data source is the critical choice :**
  - PDF documents, text files, web pages or a SQL database
  - LangChain ships ready-made loaders for each
- **Six stages, and the first four run before any question is asked :**
  - Load ; read the documents
  - Split ; cut documents too long for the model into chunks
  - Embed ; convert each chunk to a vector representing its meaning
  - Store ; persist the index so re-indexing is not needed every run
  - Retrieve ; embed the question and find the nearest chunks
  - Generate ; prompt the model with the question and those chunks
- **Embeddings come from `OpenAIEmbeddings` , `CohereEmbeddings` or Hugging Face**
- **Evaluate the result for accuracy, faithfulness to the source and latency :**
  - The stage most often skipped, and the one that catches regressions

# RAG Operation

How does Retrieval Augmented Generation work?



# Simple RAG Chain

```
docs = [
 "LangChain is a framework for working with LLM.",
 "RAG combines context retrieval with answer generation.",
 "FAISS is a library for storing and searching embeddings.",
 "Retriever is used to find the most similar documents to the user's queries. The retriever can return a variable number of documents."]
splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=20)
split = splitter.create_documents(docs)
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(split, embedding=embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})
llm = ChatOpenAI(model="gpt-5.6-luna", temperature=0)

rag_prompt = ChatPromptTemplate.from_messages([
 ("system", "Give precise answers based solely on CONTEXT. If there is no data, say don't know."),
 ("system", "CONTEXT:\n{context}"),
 ("user", "{question}")
])

rag_chain = (
 {"context": retriever, "question": RunnablePassthrough()}
 | rag_prompt | llm | StrOutputParser()
)
print(rag_chain.invoke("What is FAISS and what is it for?"))
```



# Tool Results and the Context Window

- **A tool result enters the window verbatim and stays there :**
  - A call returning 40 KB of JSON costs those tokens on every later turn
  - Three such calls can exhaust the window before the agent reaches an answer
- **Tool definitions cost tokens once per call ; tool results keep costing them**
- **Three ways to bound a tool result :**
  - Return the answer, rather than the payload the answer was taken from
  - Paginate, and let the model ask for the next page if it needs one
  - Write the payload elsewhere and return a handle to it
- **A tool should return what the model needs in order to decide, and no more**
- **Oversized tool results are the most common cause of context exhaustion in a working agent**

# Summary : Context Engineering

- The context window is a fixed budget, and the agent loop spends it
- Context engineering decides what occupies that budget on each call
- Four strategies : write, select, compress and isolate
- Memory carries earlier turns forward ; calls are stateless by default
- Full history is accurate and unbounded ; a summary bounds it for one model call
- Embeddings turn text into vectors that capture meaning rather than spelling
- Retrieval is selection : only the relevant passage enters the window
- Context failures are silent, and produce a confident answer rather than an error
- A tool result costs its tokens again on every turn that follows it

- Intro AI Agents
- LLM Fundamentals
- LangChain Intro
- LangChain Components
- ▶ **Agents and Tools**
- LangGraph Intro
- Multi Agent Systems
- Model Context Protocol
- Running Agents

- Agents
- Tools
- Examples of Tools
- Knowledgebase Tool
- Tool integrations
- TavilySearch
- YouTube Search
- Agent with Tools
- Agent Loop
- Agent Operation
- Agent without Tools
- Memory in Agents

# Agents

- **A chain is a fixed pipeline :**
  - We decide the path through the steps, and it is the same every run
- **An agent decides its own path at run time**
- **An agent is a model, a set of tools, and a loop that lets the model choose :**
  - On each pass the model picks a tool, or decides it is finished
  - Nothing outside the loop plans the sequence
- **Take the tools away and there is nothing left to decide :**
  - An agent without tools is a chain with more machinery around it
- **The loop is what makes it an agent, and the tools are what make the loop useful**

# Tools

- A model can only use what is in its prompt and what it saw during training
- That knowledge is frozen at the moment the training data was collected
- Tools break the model out of the prompt :
  - Read live data, or change something in the outside world
- In LangChain a tool is a Python function with the `@tool` decorator :
  - Three things must be clear : the arguments, the effect, and the return value
  - The model never sees the body, only the name, signature and docstring
- The docstring is therefore part of the program
- Tools range from a calculator to a search engine to an internal database

# Examples of Tools

- **Calculator :**
  - The classic first tool ; arithmetic the model would otherwise guess at
- **Search :**
  - The cheapest way to make an application feel current rather than stale
- **Databases :**
  - SQL, MongoDB or Neo4j exposed so the model can query company data
- **External APIs :**
  - Exchange rates, weather, stock prices, shipment tracking
- **The pattern is always the same :**
  - Wrap something that already works, describe it well, hand it over

# Knowledgebase Tool

- A tool is a plain function with the `@tool` decorator :
  - The docstring is what the model reads to decide when to call it

```
@tool
def company_info(name: str) -> str:
 """Returns basic company information based on name."""
 companies = {
 "OpenAI": "Artificial intelligence company, creator of ChatGPT.",
 "LangChain": "Library for building applications with language models.",
 "Tesla": "Manufacturer of electric cars and batteries."}
 return companies.get(name, "I don't know this company.")
```

- Pass the tools to the agent as a list

```
agent = create_agent(model=llm, tools=[company_info])
```

# Tool integrations

- Available tools in LangChain :

- The full list lives at

[python.langchain.com/docs/integrations/tools](https://python.langchain.com/docs/integrations/tools)

## Search

The following table shows tools that execute online searches in some shape or form:

| Tool/Toolkit                      | Free/Paid                                | Return Data                             |
|-----------------------------------|------------------------------------------|-----------------------------------------|
| <a href="#">cloro</a>             | Paid                                     | URL, Snippet, Title, Answer             |
| <a href="#">Exa Search</a>        | 1000 free searches/month                 | URL, Author, Title, Published Date      |
| <a href="#">Google Search</a>     | Paid                                     | URL, Snippet, Title                     |
| <a href="#">Linkup Search</a>     | 2000 free searches/month                 | URL, Content, Sources                   |
| <a href="#">Nia Toolkit</a>       | Free tier available                      | Code, Docs, Metadata, Sources           |
| <a href="#">Nimble Search</a>     | Free trial available                     | URL, Content, Title                     |
| <a href="#">Parallel Search</a>   | Paid                                     | URL, Title, Excerpts                    |
| <a href="#">Perplexity Search</a> | Paid (with monthly free tier)            | URL, Title, Snippet, Date, Last Updated |
| <a href="#">Tavily Search</a>     | 1000 free searches/month                 | URL, Content, Title, Images, Answer     |
| <a href="#">Apify</a>             | Free tier, pay-per-use (varies by Actor) | Actor output (varies by Actor)          |
| <a href="#">You.com Search</a>    | \$100 in credits on sign up              | URL, Title, Page Content                |



# TavilySearch

- Free internet search tool that can be used in Agents :
  - Can also be used on its own without agents

```
from langchain_tavily import TavilySearch
from langchain.agents import create_agent

Initialize search tools
Tavily API key: https://app.tavily.com/home
search = TavilySearch(max_results=3)

Direct use of tools
query = "The latest news about the UK Prime Minister."
results = search.invoke({"query": query})

TavilySearch returns a dict; the hits live under "results"
hits = results["results"] if isinstance(results, dict) else results

print(f"Search results for query: '{query}'")
for i, result in enumerate(hits, 1):
 print(f"\n{i}. {result.get('title', 'Missing title')}")
 print(f" URL: {result.get('url', 'Missing URL')}")
 print(f" Content: {result.get('content', 'Missing content')[:200]}...")
```

# YouTube Search

```
from langchain_community.tools import YouTubeSearchTool
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

youtube = YouTubeSearchTool()

llm = ChatOpenAI(model="gpt-5.6-luna", temperature=0)
agent = create_agent(
 model=llm,
 tools=[youtube],
 system_prompt="You help users find relevant YouTube videos using the youtube_search tool.",
)

result = agent.invoke(
 {"messages": [HumanMessage(content="Find some videos about LangGraph tutorials")]}
)
for m in result["messages"]:
 m.pretty_print()
```

# Agent with Tools

- Three steps : define tools, build the model, create the agent

```
from langchain.agents import create_agent
from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
 """Add two numbers together."""
 return a + b

@tool
def get_weather(city: str) -> str:
 """Get the current weather for a given city."""
 return f"It is 18 degrees and sunny in {city}."

agent = create_agent(ChatOpenAI(model="gpt-5.6-luna", temperature=0),
 [add, get_weather])

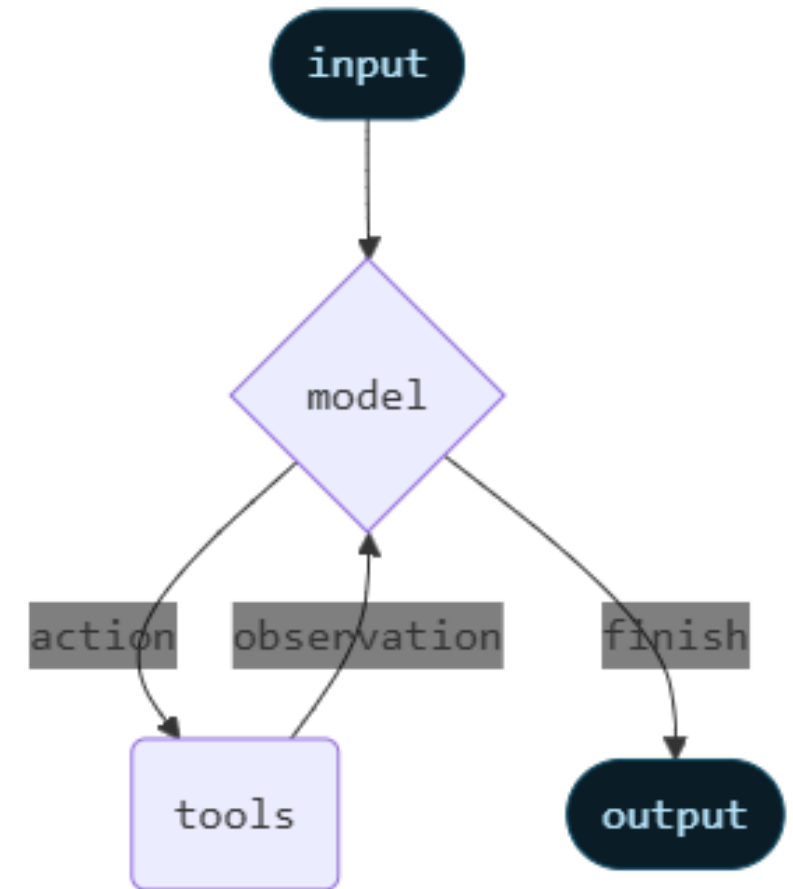
result = agent.invoke({"messages": [("human", "What is 21 + 21?")]}))
print(result["messages"][-1].content) # the final answer
```

# Agent Loop

- Each turn sends the conversation plus the tool descriptions to the model
- The reply is an `AIMessage` in one of two shapes
- It has `tool_calls` :
  - The model needs results before it can answer
  - The agent runs the tools, appends each result as a `ToolMessage`, and calls again
- It has content and no `tool_calls` :
  - The model has what it needs, and that message is the final answer
- Finishing is the absence of a tool call :
  - Nothing declares completion ; the model stops asking
- The docstrings are reread on every turn :
  - The model uses them each time to judge whether it still needs a tool

# Agent Operation

- Agent finishes when model returns response without tool calls :
  - There is no other stopping condition



# Agent without Tools

- With no tools there is nothing to decide and no loop to run
- The model reads the question and answers in one pass
- `llm.invoke()` already does that, with less ceremony

```
agent = create_agent(llm, tools=[]) # no tools
result = agent.invoke({"messages": [("human", "What is the capital of France?")]})
```

- The cost is real : message formatting, graph execution, looping logic
- Reach for an agent when the model has a choice to make

# Memory in Agents

- An agent remembers through a checkpointer, which saves state after each step
- A conversation is identified by a `thread_id` passed in `config`

```
agent = create_agent(llm, tools, checkpointer=InMemorySaver())

config = {"configurable": {"thread_id": "customer-42"}}
agent.invoke({"messages": [("human", "My name is Feiko.")]}, config)
agent.invoke({"messages": [("human", "What is my name?")]}, config)
```

- Both calls share the thread, so the second sees the first
- A new `thread_id` starts a fresh conversation with empty memory
- `InMemorySaver` keeps everything in RAM and is gone when the script ends
- `SqliteSaver` and `PostgresSaver` survive a restart
- A checkpointer is the write strategy : the history lives outside the window

# Summary : Agent and Tools

- An agent is a model, a set of tools, and a loop that lets the model choose
- A chain follows a path we fixed ; an agent picks its own at run time
- Tools break the model out of its prompt and its training cut-off
- In LangChain a tool is a function with `@tool` :
  - The model sees only its name, signature and docstring
- Each turn the model either emits `tool_calls` or gives a final answer
- Finishing is the absence of a tool call
- Memory is a checkpointer keyed by `thread_id`
- An agent with no tools is a chain with extra machinery



- Intro AI Agents
- LLM Fundamentals
- LangChain Intro
- LangChain Components
- Agents and Tools
- ▶ **LangGraph Intro**
- Multi Agent Systems
- Model Context Protocol
- Running Agents

- Why LangGraph?
- LangChain vs. LangGraph
- LangGraph Concepts
- Hello LangGraph
- Conditional Edges
- Reducers
- Simple Reducer
- Loops or Cycles
- MessagesState
- InMemorySaver
- Agent Single Pass
- Agent Single Pass
- Agent Loop
- Agent Loop
- tools\_condition
- ReAct Agent

# Why LangGraph?

- Chains suit sequential flow : `prompt -> model -> parser -> result`
- A chain is a straight line ; a graph is a network of connections
- Real workflows are rarely linear :
  - Deciding between paths, retrying until a condition holds
  - Agents coordinating in parallel, or waiting on human approval
- LangGraph supports branching and loops natively
- State is explicit and typed :
  - Every node reads and writes one shared dict, so nothing is hidden
- Human-in-the-loop is built in :
  - `interrupt_before` pauses at a node, `Command.resume` continues it

# LangChain vs. LangGraph

|                   | LangChain (LCEL)           | LangGraph                       |
|-------------------|----------------------------|---------------------------------|
| Shape             | Linear pipeline            | Directed graph (cycles allowed) |
| State             | Implicit / passed value    | Explicit typed state dict       |
| Branching         | Limited ( RunnableBranch ) | First-class conditional edges   |
| Loops             | Not supported              | Native                          |
| Human-in-the-loop | Manual                     | Built-in interrupts             |
| Best for          | Simple chains, RAG         | Agents, multi-step workflows    |

# LangGraph Concepts

- **Graph** : the workflow as a whole, the plan of action
- **State** : a `TypedDict` every node reads from and writes to
- **Node** : one step, a plain Python function taking state and returning updates :
  - It returns only the keys it changed, never the whole state
- **Edge** : a fixed connection from one node to the next
- **Conditional edge** : an edge that picks its target from the current state :
  - The router returns a node name, and the graph goes there
- **END** : the constant that stops execution
- **The graph is built, compiled once, then invoked like any callable** :
  - Compiling validates the wiring, so a missing node fails before it runs

# Hello LangGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, END

class SimpleState(TypedDict):
 value: int

def double(state: SimpleState) -> dict: # node
 return {"value": state["value"] * 2}

def add_ten(state: SimpleState) -> dict: # node
 return {"value": state["value"] + 10}

Build the graph
builder = StateGraph(SimpleState)
builder.add_node("double", double)
builder.add_node("add_ten", add_ten)

Edges
builder.set_entry_point("double")
builder.add_edge("double", "add_ten")
builder.add_edge("add_ten", END)

graph = builder.compile() # Compile

result = graph.invoke({"value": 5}) # Invoke
print(result) # {'value': 20} (5*2 + 10)
```

# Conditional Edges

- A normal edge always sends flow to one fixed next node :
  - `add_edge("a", "b")`
- A conditional edge runs a routing function at execution time :
  - It inspects the state and returns the name of the next node
  - Used to retry, fix or re-generate when an answer is wrong
- Two pieces are needed :
  - A router function taking the state and returning a string
  - An `add_conditional_edges` call that wires it up

```
def evaluate(state: State) -> dict:
 return {"verdict": "pass" if state["score"] >= 60 else "fail"}

def route(state: State) -> Literal["pass_node", "fail_node"]:
 return "pass_node" if state["verdict"] == "pass" else "fail_node"

builder.add_conditional_edges("evaluate", route)
```

# Reducers

- A reducer is a merge policy attached to one key of the state
- It answers a single question :
  - When a node returns a new value for this field, overwrite or combine?
- Without a reducer the default is overwrite
- Overwrite is right for a counter and wrong for a message list :
  - Overwriting would throw away the conversation and keep the last message only
- Attach one with `Annotated`, pairing the field's type with a merge function

```
class State(TypedDict):
 messages: Annotated[list, add_messages] # append
 total: int # overwrite
```

- `add_messages` appends, and `operator.add` concatenates lists or strings

# Simple Reducer

- An `Annotated` field with a reducer merges updates instead of overwriting

```
class State(TypedDict):
 collected: Annotated[list[str], operator.add] # reducer: concatenate
 last: str # no reducer: overwrite

def node_a(state: State) -> dict:
 return {"collected": ["a"], "last": "a"}

def node_b(state: State) -> dict:
 return {"collected": ["b"], "last": "b"}

result = graph.invoke({"collected": [], "last": ""})
print(result) # {'collected': ['a', 'b'], 'last': 'b'}
```



# Loops or Cycles

```

GraphRecursionError Traceback (most recent call last)
Cell In[12]: line 27
```

- There is no loop construct ; a loop is an edge pointing back to an earlier node
- A conditional edge is what makes it terminate :
  - Its router decides between going around again and exiting
- Progress comes from state :
  - Each pass merges updates through the reducer, so the exit test can change
- A loop whose state never changes will not stop on its own
- The recursion limit is the safety net, and it raises rather than hanging

# MessagesState

- `MessagesState` is a prebuilt `TypedDict` with one key already declared
- It saves rewriting the same annotation in every chat or agent graph

```
class MessagesState(TypedDict):
 messages: Annotated[list[AnyMessage], add_messages]
```

- `add_messages` does more than `operator.add` :
  - It deduplicates by message ID rather than blindly appending
  - Every LangChain message carries an `id`
  - An incoming message whose ID already exists replaces that entry
- Replacement is what makes editing and re-running a turn work
- Using it is identical to declaring the field by hand

# InMemorySaver

- A checkpointer saves graph state at every superstep, keyed by thread ID
- The checkpointer gives a graph memory between separate `.invoke()` calls
- It also enables human-in-the-loop, time travel and fault recovery
- `InMemorySaver` keeps checkpoints in a dict in RAM :
  - Fine for debugging and tests, gone when the process exits

```
graph = builder.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "conversation-1"}}
graph.invoke({"messages": [("human", "Hello")]}, config)
```

- For production, `PostgresSaver` or `SqliteSaver` swap in on one line

# Agent Single Pass

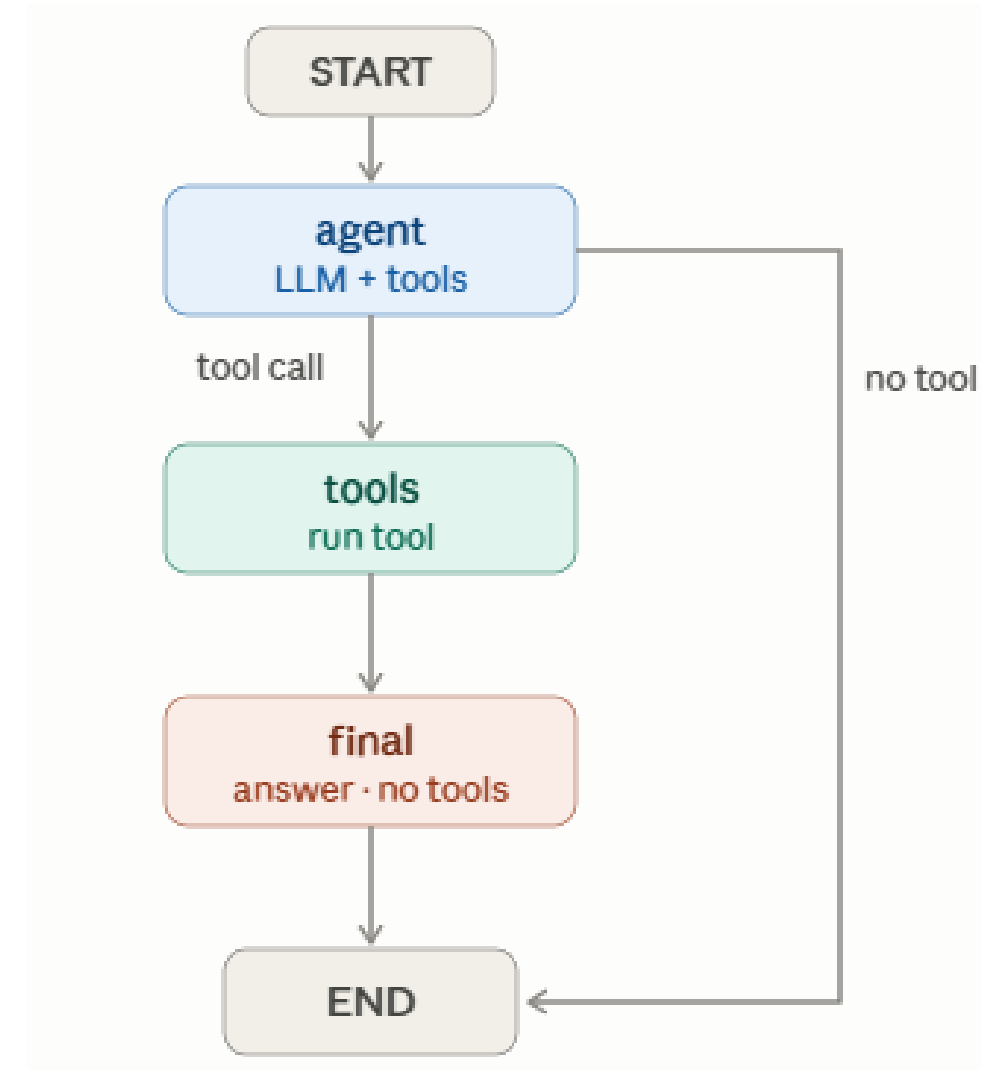
- The model chooses tools, but gets exactly one turn :
  - Tools are plain functions wrapped with `@tool`
  - After they run, flow goes forward to a final node instead of looping back
- `ToolNode` is the graph node that runs whatever tools the model asked for

```
builder.add_node("agent", agent)
builder.add_node("tools", ToolNode(tools))
builder.add_node("final", final_answer)

builder.set_entry_point("agent")
builder.add_edge("agent", "tools")
builder.add_edge("tools", "final")
builder.add_edge("final", END)
```

# Agent Single Pass

- **Everything flows strictly downward :**
  - The agent gets exactly one turn to call tools
  - Whatever it retrieves goes forward to final which has no tools bound and then to END



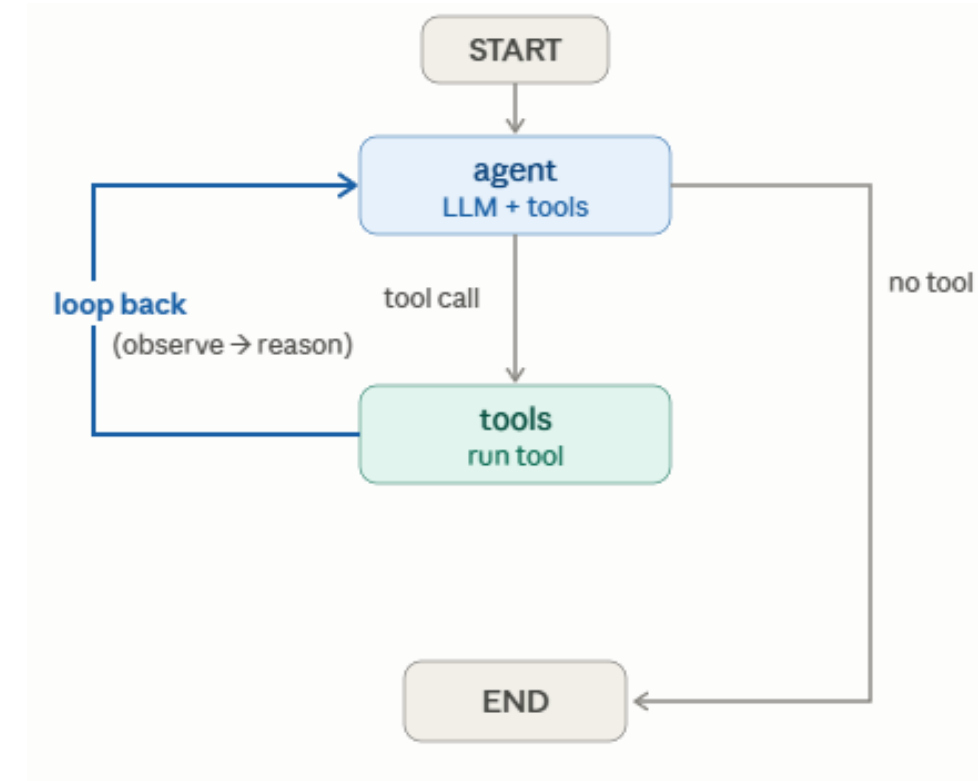
# Agent Loop

- A single pass handles at most one step of a chain
- A loop gives a fresh turn each time a tool result comes back :
  - The chain can then be as long as the task needs
- Two edges make the difference :
  - `tools` goes back to `agent` instead of forward to a final node
  - `agent` gets a conditional edge that can route to `END`

```
builder.add_edge("tools", "agent") # loop back
builder.add_conditional_edges("agent", tools_condition)
```

# Agent Loop

- Tools no longer flow into a dead-end final node :
  - They loop back into the agent, carrying the observation
- The exit is the no-tool branch :
  - When the agent stops emitting tool calls, `tools_condition` routes to `END`



# tools\_condition

- A prebuilt router deciding whether the loop continues after the agent runs
- It reads the last message in `state["messages"]` :
  - The last message is the `AIMessage` the agent just produced
- The decision is one check :
  - `tool_calls` present, return `"tools"` and run them
  - `tool_calls` absent, return `END` and stop
- Written out by hand it is four lines

```
def tools_condition(state) -> Literal["tools", "__end__"]:
 last = state["messages"][-1]
 if getattr(last, "tool_calls", None):
 return "tools"
 return END
```

- Nothing declares completion ; the absence of a tool call is the signal



# ReAct Agent

- **ReAct is Reasoning plus Acting, from a 2022 paper :**
  - Models solve tool-using tasks better when reasoning interleaves with acting
- **The loop has three beats :**
  - Thought ; reason about what is needed next
  - Action ; call a tool
  - Observation ; read the result and return to Thought
- **The graph is `agent -> tools_condition -> tools -> agent`**
- **`create_agent` collapses that whole graph into one call**

```
agent = create_agent(
 model=llm, # create_agent binds the tools itself
 tools=[find_station, read_station],
 system_prompt="Call find_station for the city, then read_station with "
 "the returned ID. Never invent a station ID or a temperature.")
```

# Summary : LangGraph Intro

- Real workflows are rarely linear ; an agent picks between paths
- LangGraph models a workflow as a directed graph, giving full control
- A conditional edge calls a router that names the next node
- There is no special loop construct :
  - A loop is an edge pointing back to a node already visited
- **InMemorySaver** is the simplest checkpointer :
  - Saves state at every superstep, keyed by thread ID
- A single agent pass allows one tool-calling turn
- An agent loop gives a fresh turn per tool result
- The prebuilt ReAct agent collapses that graph into one call

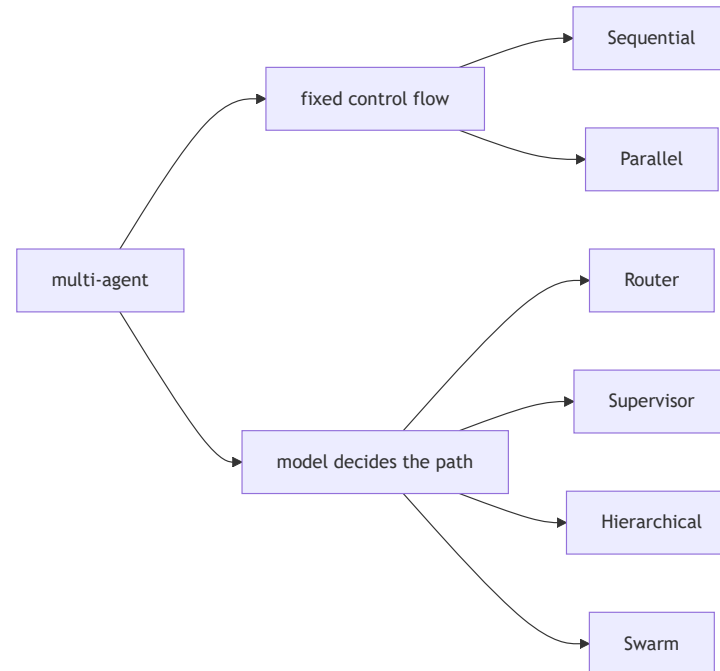
- Intro AI Agents
- LLM Fundamentals
- LangChain Intro
- LangChain Components
- Agents and Tools
- LangGraph Intro
- ▶ **Multi Agent Systems**
- Model Context Protocol
- Running Agents

- Multi Agent Systems
- Multi Agent Patterns
- Sequential Pattern
- Stage One
- Stage Two and Three
- Router Pattern
- Task Classification
- Supervisor Pattern
- Wiring Steps
- Create Supervisor
- Swarm Pattern
- Create Swarm
- Create Handoffs
- Configure Model
- Dynamic Model
- Dynamic System Prompt
- Thirty Thousand Theorems
- What the Fermat Project Shows

# Multi Agent Systems

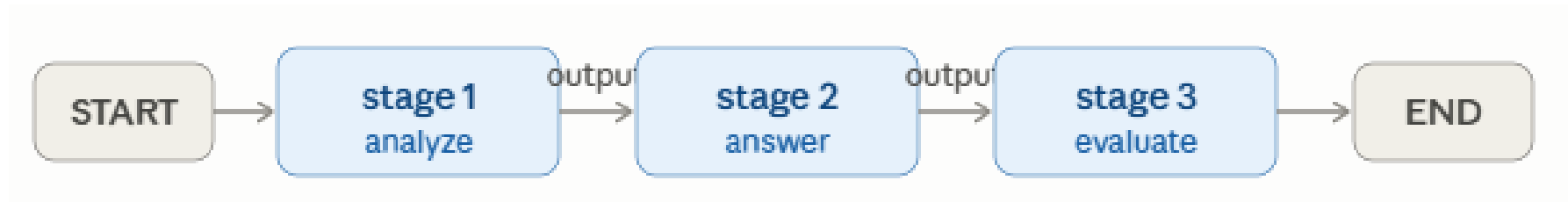
- One well-designed agent with a good toolset goes a long way
- Multi-agent architectures cost latency, tokens and debugging effort
- Three problems justify paying that cost
- **Tool overload :**
  - A single agent binds every tool on every call
  - The definitions eat context, and the model chooses among all of them each turn
- **Prompt focus :**
  - A prompt trying to be researcher, coder and writer conflicts with itself
  - Models follow one narrow, coherent instruction far better
- **Context partitioning :**
  - A sub-agent can make twenty noisy tool calls and return a clean summary
  - The orchestrator's context stays small

# Multi Agent Patterns



- Sequential : output feeds the next agent
- Parallel : run concurrently, merge the results
- Router : one decision, then one specialist runs to **END**
- Supervisor : a hub delegates and gets control back
- Hierarchical : supervisors of supervisors
- Swarm : peers hand off, no coordinator

# Sequential Pattern



- The simplest arrangement : a fixed pipeline of specialists
- Each agent's output is the next agent's input, like an assembly line
- Analyse the data, generate an answer, evaluate it
- There are no routing decisions ; the order is hard-coded at build time
- It is the multi-agent version of the single-pass agent :
  - Several specialists in a row, still no loop and no branching
- Works when the process is well understood and does not vary

# Stage One

- Agent 1 analyzes a complaint type :

```
class State(TypedDict):
 message: str # the customer message (input)
 category: str # stage 1 (LLM) writes this
 priority: str # stage 2 (deterministic) writes this - depends on category
 reply: str # stage 3 (LLM) writes this - prompt depends on category

Stage 1 - LLM classifies the message into one fixed label
def classify(state: State) -> dict:
 prompt = (
 "Classify the customer message as exactly one word: "
 "complaint, question, or praise. Reply with only that word.\n\n"
 f"Message: {state['message']}"
)
 raw = llm.invoke(prompt).content.strip().lower()
 # normalise to a known label (LLMs don't always return exactly one word)
 category = next((c for c in ("complaint", "question", "praise") if c in raw), "question")
 return {"category": category}
```

- Return of classify is a partial State update for the field category :

```
return {"category": category}
```

# Stage Two and Three

- Agent 2 determines priority based on `category` field :
  - Agent 3 write a short story base on the priority field :

```
Stage 2 - DETERMINISTIC branch: priority is chosen from stage 1's label
def prioritize(state: State) -> dict:
 priority = {"complaint": "HIGH", "question": "NORMAL", "praise": "LOW"}[state["category"]]
 return {"priority": priority}

Stage 3 - LLM again, but the INSTRUCTION it gets depends on the category
def respond(state: State) -> dict:
 instructions = {
 "complaint": "Write a short, apologetic reply and promise a follow-up.",
 "question": "Write a short, helpful reply that answers the question.",
 "praise": "Write a short, warm reply thanking the customer.",
 }[state["category"]]
 prompt = f"{instructions}\n\nCustomer message: {state['message']}"
 return {"reply": llm.invoke(prompt).content.strip()}
```

- Following is a dictionary lookup compressed in one line :

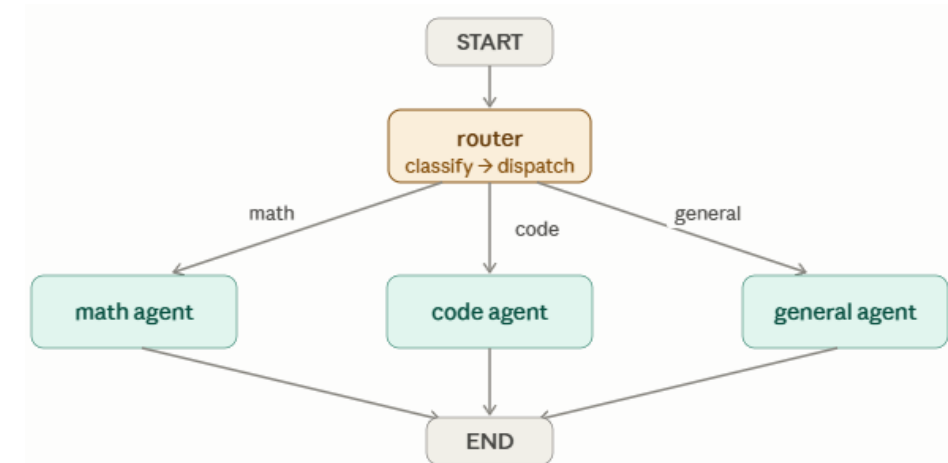
```
priority = {"complaint": "HIGH", "question": "NORMAL", "praise": "LOW"}[state["category"]]
```

- Does the same job as an `if/elif` chain, more compactly
  - Agent 3 could make phone call for "High", email for "Normal" and log entry for "Low"



# Router Pattern

- **Router pattern has one defining move :**
  - Single routing decision sends task to one specialist and that specialist handles it
- **Unlike the sequential pipeline where every stage runs in order :**
  - Router fires once and forwards
  - Dispatcher classifies the call, connects to one expert and steps out
- **Route to one of several is exactly what conditional edge expresses :**
  - The pattern is one `add_conditional_edges` call



# Task Classification

- The router classifies the task but does not answer it

```
def router(state: State) -> dict:
 prompt = ("Classify the question into exactly one word: "
 f"math, code, or general.\n\nQuestion: {state['question']}")
 raw = llm.invoke(prompt).content.strip().lower()
 category = next((c for c in ("math", "code", "general") if c in raw), "general")
 return {"category": category}

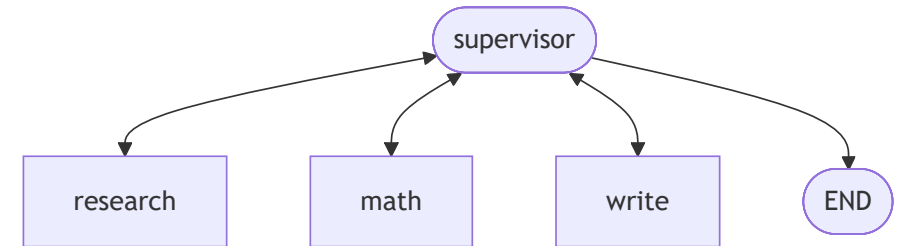
def route(state: State) -> str:
 return state["category"]
```

- Each specialist runs once, then goes straight to **END**

```
builder.add_conditional_edges("router", route,
 {"math": "math", "code": "code", "general": "general"})
```

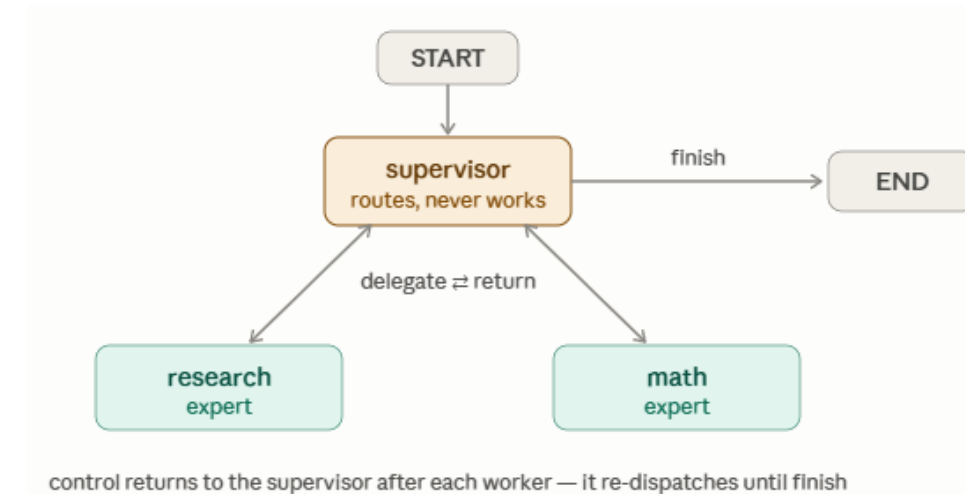
# Supervisor Pattern

- A central orchestrator coordinates a team of specialist workers
- It receives the task, delegates the next step, and gets control back each time a worker finishes
- It does no work itself ; it only routes
- Workers never talk to each other or to the user, which is what makes it debuggable
- Unlike a router that dispatches once, the supervisor can dispatch again
- A worker is also isolation : its tool calls never enter the supervisor's window



# Wiring Steps

- The supervisor picks a worker by reading the conversation history
- Each worker runs its own ReAct agent with its own tools :
  - It appends a tagged message and hands control back
- Three pieces of wiring :
  - Conditional edges out of the supervisor, to a worker or to `END`
  - A plain edge from every worker back to the supervisor
  - A `route()` function reading `next_agent` from state



# Create Supervisor

- `create_supervisor` from `langgraph-supervisor` delegates to named workers

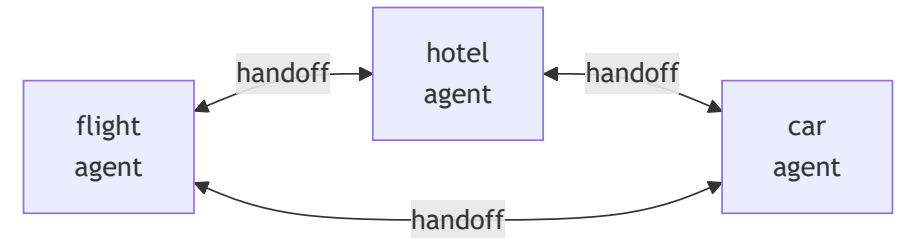
```
supervisor = create_supervisor(
 [research_agent, math_agent],
 model=model,
 prompt="You manage a research_expert and a math_expert. "
 "Delegate to one at a time and do no work yourself. "
 "When the task is complete, give the final answer and stop.",
).compile()
```

- Each worker is a ReAct agent with its own tools and name

```
research_agent = create_agent(model, tools=[web_search],
 name="research_expert", system_prompt="Find facts with web_search. Do no math.")
```

# Swarm Pattern

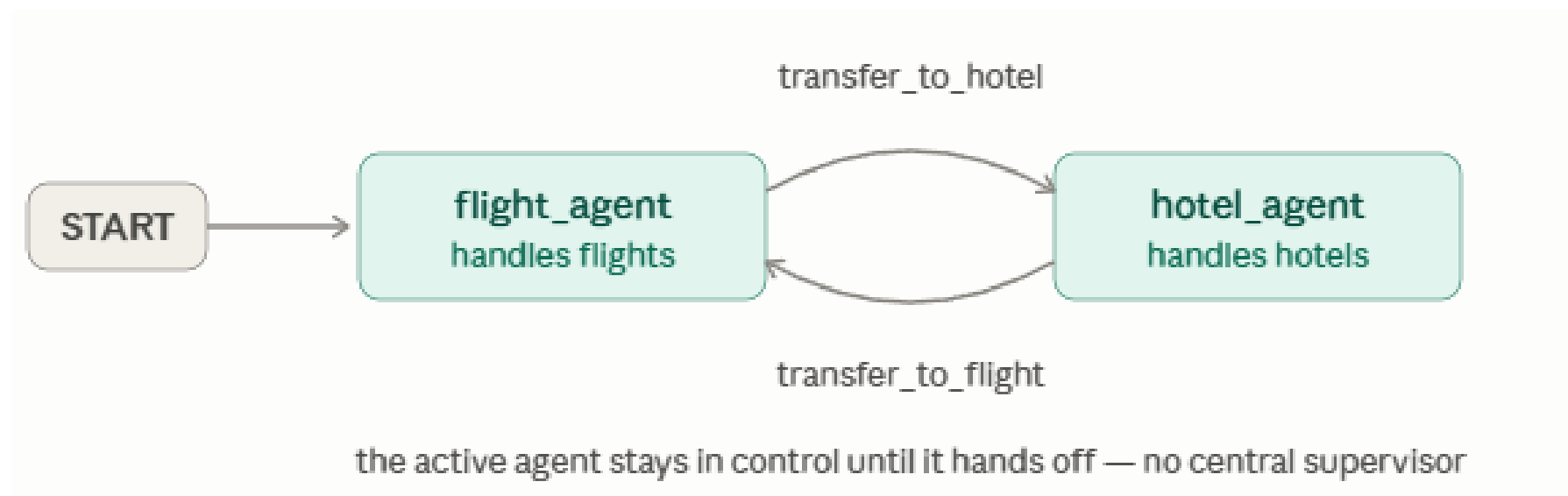
- **Decentralised** : specialists hand control to each other, with no coordinator
- **Each agent decides when its part is done and which peer takes over**
- **Transfers happen through handoff tools** :
  - A tool that passes control and the shared message history to a named peer
- **State tracks an active agent, so the last one to receive control takes the next turn**
- **A checkpointer is required** :
  - Without it the swarm forgets who was active between turns
- **Routing stays predictable because each handoff tool names exactly one peer**
- **Suits conversational assistants where a user stays with one expert until they need another**



# Create Swarm

- **LangGraph ships prebuilt implementation `create-swarm` :**
  - Need to import it from `langgraph-swarm` library
- **Assemble swarm by giving each agent handoff tools :**
  - With `create_handoff_tool` and listing them together
- **Checkpointers REQUIRED so swarm remembers active agent across turns :**

```
swarm = create_swarm(
 [flight_agent, hotel_agent],
 default_active_agent="flight_agent",
).compile(checkpointer=InMemorySaver())
```



# Create Handoffs

- Each agent carries a handoff tool naming the other agent

```
handoff_to_hotel = create_handoff_tool(agent_name="hotel_agent")
flight_agent = create_agent(model, tools=[book_flight, handoff_to_hotel],
 name="flight_agent")
```

- Turn 1 starts on `flight_agent`, the default active agent
- Turn 2 on the same thread asks about hotels :
  - `flight_agent` hands off to `hotel_agent`
  - `hotel_agent` stays active for the rest of the conversation



# Configure Model

- A tool's name, description and argument names all become part of the prompt
- `@tool` adds that metadata and enables runtime injection via `ToolRuntime`
- **Model parameters are chosen per use case :**
  - Low temperature for routing and classification
  - A larger model for the step that reasons

```
llm = ChatOpenAI(model="gpt-5.6-luna", temperature=0)
agent = create_agent(llm, tools, checkpointer=InMemorySaver())
```

- **The checkpointer is what carries state across interactions**

# Dynamic Model

- A dynamic model is selected at runtime from the current state :
  - Supports routing logic and cost optimisation
  - Built as middleware with the `@wrap_model_call` decorator

```
@wrap_model_call
def choose_model(request, handler):
 if len(request.state["messages"]) > 10:
 request.model = big_model
 return handler(request)
```

# Dynamic System Prompt

- The prompt itself can be built per request from context

```
class Context(TypedDict):
 user_role: str

@dynamic_prompt
def user_role_prompt(request: ModelRequest) -> str:
 role = request.runtime.context.get("user_role", "user")
 if role == "expert":
 return "You are a helpful assistant. Give detailed technical answers."
 if role == "beginner":
 return "You are a helpful assistant. Explain simply, avoid jargon."
 return "You are a helpful assistant."

agent = create_agent(model="openai:gpt-5.6-sol", tools=[web_search],
 middleware=[user_role_prompt], context_schema=Context)

agent.invoke({"messages": [("human", "Explain machine learning")]},
 context={"user_role": "expert"})
```

# Thirty Thousand Theorems

- **In 2026 Claude formalised the proof of Fermat's Last Theorem in Lean :**
  - Wiles proved it in 1995 ; this made the proof machine-checkable
  - It rests on nothing beyond Lean's three axioms
- **Eleven days of largely autonomous work :**
  - 30,300 theorems proved, 29,500 of them used in the final proof
  - 13 million lines of Lean, more than five times the size of Mathlib
  - Roughly 6 billion output tokens
- **Dozens of agents worked in parallel ; this was not one long-running agent**
- **The humans steered rather than drove :**
  - Occasional instructions of the order of "push Mazur to be done soon"
  - They reviewed the result ; they did not write the proofs

# What the Fermat Project Shows

- **The early attempts failed for reasons other than model strength :**
  - The agents lost track of the project state and stopped collaborating
- **The fix sat outside the model, in a shared dependency graph of theorems :**
  - Statements kept separate from proofs, each with a description in words
  - Any agent could see what was proved, what was claimed and what was left
- **The blueprint is external shared state, which is what a supervisor gives us**
- **Verification is what made the autonomy affordable :**
  - Lean's type checker accepts no step that does not follow
  - A comparator confirmed the statement matched Mathlib's official one
- **An agent whose work can be checked mechanically can be left to run**
- **About 7% of the work went down paths that were abandoned :**
  - Normal for an agent, and a cost to plan for rather than a defect

# Summary : Multi Agent Systems

- One good agent goes a long way ; reach for a team only when it earns its cost
- Tool overload, prompt conflict and context bloat are the three reasons
- Sequential : a fixed pipeline of specialists, no routing at all
- Router : one decision, then the chosen specialist runs to **END**
- Supervisor : control returns to the hub after every worker :
  - Workers never talk to each other, which is what makes it debuggable
- Swarm : peers hand off directly, and a checkpointer tracks who is active
- Handoffs name exactly one peer, so the possible transfers stay bounded
- Models and prompts can be chosen per request through middleware
- Fermat shows the two things a long run needs : shared state, and a checker

- Intro AI Agents
- LLM Fundamentals
- LangChain Intro
- LangChain Components
- Agents and Tools
- LangGraph Intro
- Multi Agent Systems
- ▶ **Model Context Protocol**
- Running Agents

- What is MCP?
- The N x M Problem
- Where MCP Came From
- The Three Roles
- Server Primitives
- An MCP Tool Versus a LangChain Tool
- MCP Is Stateless
- Transports
- Discovery and Caching
- Building a Server
- Resources and Prompts
- Running the Server
- Calling a Server
- Consuming MCP from the Claude API
- Consuming MCP from the OpenAI Agents SDK
- Servers Worth Connecting To
- Connecting One
- Tool Filtering and Scale
- How MCP Servers Get Abused
- The State of Deployed Servers
- Securing a Server
- The Ecosystem

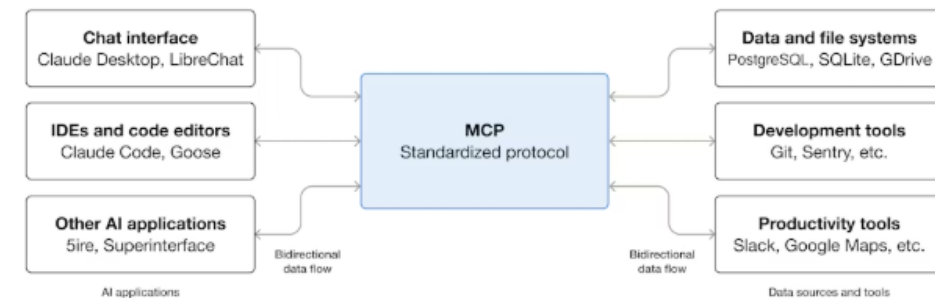
# What is MCP?

- An open protocol for connecting models to the systems they act on
- It standardises one thing : how a model discovers and calls external capabilities
- A server exposes capabilities ; a client consumes them :
  - Neither needs to know anything about the other beyond the protocol
  - A server written once works with every client that speaks MCP
- It does not make a model smarter ; it makes integrations reusable :
  - The same weather server serves our agent, an IDE and a chat app unchanged



# The N x M Problem

- **Before MCP, every integration was bespoke :**
  - Each application wrote its own connector for each data source
  - N applications and M sources meant N times M connectors
- **Each one reimplemented the same concerns :**
  - Describing the capability, validating arguments, handling auth and errors
- **MCP turns that into N plus M :**
  - Every client speaks one protocol, every server implements one protocol
- **The same argument USB made for peripherals, applied to tool access**

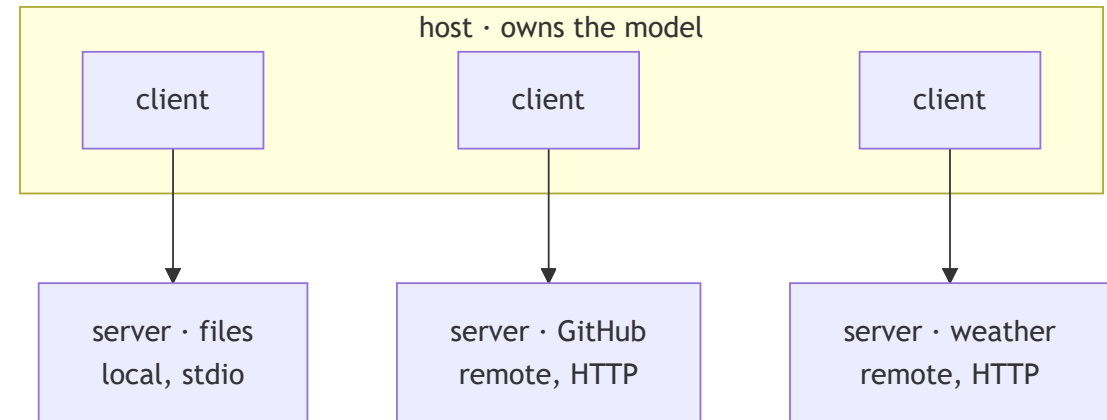


# Where MCP Came From

- Anthropic published the first specification in November 2024
- Adoption moved unusually fast for a protocol :
  - OpenAI adopted it in the Agents SDK in March 2025
  - Google committed Gemini in April 2025
  - Microsoft made it first-class across its platforms at Build 2025
- Anthropic donated MCP to the Agentic AI Foundation at the Linux Foundation in December 2025
- It is no longer any one vendor's protocol, and that matters when we bet on it
- Changes go through a public SEP process with a twelve-month deprecation window

# The Three Roles

- **Host : the application the user interacts with :**
  - An IDE, a chat interface, an agent we wrote
  - It owns the model and decides what it is allowed to do
- **Client : the connector inside the host, one per server**
- **Server : the process exposing capabilities :**
  - Local as a subprocess, or remote behind a URL
- **The split matters for trust :**
  - A server never sees the conversation, only the arguments of its own calls
  - The host decides which servers exist and which tools are offered



# Server Primitives

- A server can expose three kinds of thing, and they differ by who is in control
- **Tools** : functions the model chooses to call :
  - Model-controlled, and the kind agents use
- **Resources** : data the host can read, addressed by URI :
  - Application-controlled ; the host decides what to load into context
- **Prompts** : templates the user can invoke deliberately :
  - User-controlled ; typically surfaced as slash commands
- Most servers in the wild implement tools and nothing else
- Anthropic's own connector supports only tools, which tells us where the weight is

# An MCP Tool Versus a LangChain Tool

- Functionally they are the same idea : a described function the model may call
- A `@tool` in LangChain lives inside our process :
  - It ships with our application and is available only to it
- An MCP tool lives behind a protocol boundary :
  - It runs in another process, possibly written by someone else
  - Any MCP client can reuse it, beyond the agent it was written for
  - It can be updated without redeploying the application that uses it
- The cost is a network hop, a schema round-trip and an extra thing to secure
- Use MCP when the capability outlives one application

# MCP Is Stateless

- Since 2026-07-28 there is no handshake and no session :
  - The first message a client sends can be a real request
- Every request carries its own version and capabilities in `_meta`

```
"_meta": {
 "io.modelcontextprotocol/protocolVersion": "2026-07-28",
 "io.modelcontextprotocol/clientCapabilities": {},
 "io.modelcontextprotocol/clientInfo": { "name": "course-demo", "version": "1.0.0" }
}
```

- A server must not infer anything from earlier requests on the same connection
- Any request can land on any instance behind a load balancer :
  - A remote MCP server is now an ordinary HTTP workload

# Transports

- **stdio : the server is a subprocess, messages over stdin and stdout :**
  - The default for local servers ; credentials come from the environment
- **Streamable HTTP : one endpoint, POST for requests, streamed responses :**
  - The transport for anything remote
- **The older HTTP+SSE transport is deprecated**
- **HTTP requests must carry `Mcp-Method` and `Mcp-Name` headers :**
  - Gateways can route, rate-limit and meter without parsing the body
- **Authorization applies to HTTP only, built on OAuth 2.1 :**
  - stdio servers take credentials from the environment instead

# Discovery and Caching

- `server/discover` reports supported versions, capabilities and identity :
  - Clients may call it first, or skip straight to a real request
- List results carry cache hints so clients stop re-fetching them

```
{ "result": { "resultType": "complete", "tools": [...],
 "ttlMs": 300000, "cacheScope": "private" } }
```

- `ttlMs` is a freshness hint ; `cacheScope` says whether a shared proxy may store it
- Servers should return tools in a deterministic order :
  - A stable tool list keeps the model's prompt cache warm



# Building a Server

- The Python SDK generates the schema from type hints and the description from the docstring

```
from mcp.server import MCPServer

mcp = MCPServer("weather", version="1.0.0")

@mcp.tool()
def find_station(city: str) -> str:
 """Return the weather station ID for a city."""
 return {"Utrecht": "NL-260", "Houten": "NL-348"}.get(city, "UNKNOWN")
```

- That docstring is the entire basis on which the model decides to call this
- The generated schema requires `city` as a string, because the annotation said so

# Resources and Prompts

- A resource is addressed by URI, and the URI may be a template

```
@mcp.resource("station://{station_id}")
def station_info(station_id: str) -> str:
 """Metadata for one weather station."""
 return f"Station {station_id}, automatic, hourly readings"

@mcp.prompt()
def daily_briefing(city: str) -> str:
 """Ask for a plain-language weather briefing."""
 return f"Summarise today's weather for {city} in two sentences."
```

- The host reads resources deliberately ; the model does not choose them
- Prompts are offered to the user, typically as a command

# Running the Server

- One object, two deployment shapes

```
if __name__ == "__main__":
 mcp.run(transport="stdio") # local subprocess
 # mcp.run(transport="streamable-http") # remote HTTP service
```

- Nothing about the tools changes between them
- During development `mcp dev server.py` runs it with the inspector attached
- The SDK ships as `mcp` on PyPI :
  - Version 2 implements 2026-07-28 ; version 1 implements the stateful revision
  - Pin deliberately, because the APIs differ

# Calling a Server

- A client connects to a server object, a command, or a URL

```
import asyncio
from mcp import Client

async def main():
 async with Client("http://localhost:8000/mcp") as client:
 tools = await client.list_tools()
 print([t.name for t in tools.tools])

 result = await client.call_tool("find_station", {"city": "Utrecht"})
 print(result.structured_content) # {'result': 'NL-260'}

asyncio.run(main())
```

- On the wire the field is `structuredContent` ; the Python object exposes `structured_content`

# Consuming MCP from the Claude API

- Remote servers can be attached to a request without writing a client

```
response = client.beta.messages.create(
 model="claude-opus-5",
 max_tokens=1000,
 messages=[{"role": "user", "content": "Which station serves Utrecht?"}],
 mcp_servers=[{"type": "url", "url": "https://example.com/mcp",
 "name": "weather", "authorization_token": TOKEN}],
 tools=[{"type": "mcp_toolset", "mcp_server_name": "weather"}],
 betas=["mcp-client-2025-11-20"],
)
```

- Both halves are required ; a server with no matching toolset is rejected
- Tool calls only, HTTPS only, no local stdio servers

# Consuming MCP from the OpenAI Agents SDK

- The same servers, a different client, because the protocol is the same

```
from agents.mcp import MCPServerStdio

async with MCPServerStdio(
 params={"command": "python", "args": ["server.py"]},
 cache_tools_list=True,
) as server:
 agent = Agent(name="Assistant", mcp_servers=[server])
```

- `MCPServerStdio`, `MCPServerStreamableHttp` and `HostedMCPTool` cover the transports
- `cache_tools_list` avoids re-listing tools on every run
- Codex can itself run as an MCP server, which is how coding agents get composed

# Servers Worth Connecting To

- Most of the value in MCP now comes from servers somebody else runs
- The vendors publish and host their own, secured with OAuth over HTTPS :
  - GitHub, Notion, Linear, Sentry, Slack, Atlassian, Stripe, Figma, Supabase
- The registries hold far more than the official one, and most of it is noise :
  - Several unofficial catalogues list tens of thousands of community servers
  - The vendor's own server is almost always the one we want
- Remote is the default now : 59% of builders use Streamable HTTP, 34% stdio
- Every one of these answers an unauthenticated call the same way :

```
POST https://mcp.notion.com/mcp -> 401
```

- Nothing is reachable until somebody signs in

# Connecting One

- Adding a server is one command, and no credential enters the config file :

```
claude mcp add -s user -t http github https://api.githubcopilot.com/mcp/
```

- `-t http` marks it a remote endpoint rather than a local process to launch
- The first call opens a browser, we sign in, and the client stores the token
- Four questions before connecting one inside a bank :
  - Who runs it, and is it the vendor's own server or somebody's copy
  - What the consent screen asks for, which is rarely the minimum
  - What it can reach : a server inherits the access of whoever signed in
  - Whether the tool descriptions have been read, because they are instructions
- Connecting a server is a delegation of authority



# Tool Filtering and Scale

- Every enabled tool costs context on every single turn
- A dozen servers can put hundreds of tool definitions in front of the model
- Two ways to cut that down :
  - Allowlist : disable by default, enable the few tools we want
  - Denylist : enable by default, disable the destructive ones
- Denylisting writes is the cheapest way to build a read-only assistant

```
{ "type": "mcp_toolset", "mcp_server_name": "calendar",
 "default_config": { "enabled": false },
 "configs": { "search_events": { "enabled": true } } }
```

- Deferred loading hides descriptions until a tool search surfaces them
- Deferred loading is selection applied to tools rather than to documents

# How MCP Servers Get Abused

- The model reads tool descriptions as instructions, and that is the attack surface
- Tool poisoning : hidden instructions in a description or its metadata :
  - The user sees a tool named `get_weather` ; the model sees the extra sentence
- Confused deputy : the server holds credentials the user does not :
  - A crafted argument makes it act with authority the caller never had
- Token passthrough : a server forwards the user's token to a downstream API :
  - This breaks OAuth's audience restriction and makes one breach total
- None of these need a protocol flaw ; they exploit the trust model working normally

# The State of Deployed Servers

- A 2026 audit of public MCP servers found the ecosystem in poor shape :
  - 40% required no authentication at all
  - 43% carried command-injection vulnerabilities
  - 79% handled credentials in plaintext
- The NSA published formal MCP security guidance in June 2026
- Treat a third-party server the way we would treat a dependency we cannot read :
  - It executes with our credentials, on our data, on the model's say-so
- The registry lists thousands of servers ; almost none have been audited

# Securing a Server

- **Validate every argument :**
  - A tool is a remote code path reachable by anyone who can call the server
  - Command injection was found in 43% of audited servers, which is how
- **Never forward a received token to a downstream API :**
  - Exchange it for one scoped to what this server needs
  - Check the audience claim names this server before trusting it
- **Block outbound requests to internal addresses, or a tool becomes an SSRF probe**
- **Scope narrowly : one server, one job, the least authority that works**
- **Require human confirmation for anything destructive or irreversible**
- **Pin which servers a host may load. Discovery is not authorisation**

# The Ecosystem

- The official registry listed roughly 18,000 servers by mid-2026 :
  - Up from about 6,800 at the end of 2025, across all registries
- Python and TypeScript SDKs have each passed a billion downloads
- Capabilities beyond the core now ship as extensions :
  - `io.modelcontextprotocol/tasks` for work that outlives a request
  - MCP Apps, the first official extension, for server-supplied interfaces
- Extensions are negotiated per request through the capabilities field
- Judge a server the way we judge a library :
  - Who maintains it, how it authenticates, what it is allowed to reach

# Summary : Model Context Protocol

- MCP standardises how a model discovers and calls external capabilities
- It turns N times M bespoke integrations into N plus M
- Servers expose tools, resources and prompts ; tools carry almost all the weight
- Since 2026-07-28 the protocol is stateless, and remote servers are the default
- The vendors run their own, and OAuth is what gates them
- Connecting a server delegates our access to it, so read what it asks for
- A tool description is executable trust, which is where the attacks live
- The protocol is governed by the Linux Foundation

- Intro AI Agents
- LLM Fundamentals
- LangChain Intro
- LangChain Components
- Agents and Tools
- LangGraph Intro
- Multi Agent Systems
- Model Context Protocol
- ▶ **Running Agents**

- Running Agents
- Why Agents Fail Differently
- Common Failure Modes
- Bounding the Loop
- Evaluating an Agent
- String Evaluators
- Embedding Distance
- Comparison Evaluators
- Criteria Evaluators
- LLM as Judge
- Trajectory Evaluation
- Building an Eval Set
- Observability
- Tracing with LangSmith
- Reading a Trace
- Guardrails
- Input Guardrails
- Output Guardrails
- Human in the Loop
- Cost and Token Budgets
- Cost at Volume
- Caching
- Deployment Shapes
- Writing the Loop by Hand
- What a Harness Adds
- Designing the Tool List

# Running Agents

- Everything so far has been about building an agent
- This module is about what happens once it works on our own machine
- A working agent is not yet a deployed one :
  - It has to be measured, watched, bounded and paid for
- Four questions run through this module :
  - Evaluation : is it right
  - Observability : what did it do
  - Guardrails : what is it allowed to do
  - Budgets and caching : what does it cost
- All four have direct support in LangChain and LangGraph



# Why Agents Fail Differently

- A program with a bug throws, or returns the wrong value every time
- An agent is non-deterministic :
  - The same input can take a different path on the next run
  - A test that passed once therefore says very little
- The failures are usually silent :
  - The agent returns a confident, well-formatted, wrong answer
  - From outside the model, a hallucination looks exactly like a good answer
  - Nothing raises, and no status code says anything is amiss
- The path matters as much as the answer :
  - An agent can reach the right result through the wrong tool calls
  - That works today and breaks the moment the data changes
- The question is never whether it crashed, but whether it was right, and how it got there

# Common Failure Modes

- **Loops :**
  - The model calls the same tool with the same arguments over and over
  - Usually because the tool result does not answer the question it was asked
- **Tool thrash :**
  - The model cycles between tools without converging on an answer
  - A reliable sign that two tool descriptions overlap
- **Context exhaustion :**
  - The accumulated history outgrows the context window part way through a run
  - Older messages fall out, and the agent forgets its own instructions
- **Invented tool arguments :**
  - A well-formed call carrying a value the model made up
- **Runaway cost :**
  - None of the above raises anything ; the bill is the first symptom

# Bounding the Loop

- LangGraph stops a run that will not terminate :

```
graph.invoke({"n": 0}, {"recursion_limit": 10})
```

GraphRecursionError: Recursion limit of 10 reached without hitting a stop condition. You can increase the limit by setting the `recursion\_limit` config key.

- Middleware bounds the agent rather than the graph :

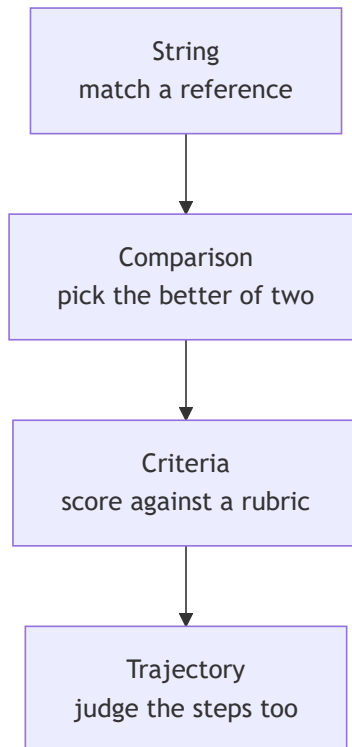
```
from langchain.agents.middleware import (
 ModelCallLimitMiddleware, ToolCallLimitMiddleware)

agent = create_agent(model=llm, tools=tools, middleware=[
 ModelCallLimitMiddleware(run_limit=8, exit_behavior="end"),
 ToolCallLimitMiddleware(run_limit=6, exit_behavior="end")])
```

- `exit_behavior="end"` returns whatever the agent has ; `"error"` raises instead

# Evaluating an Agent

- Evaluation answers one question : is the output good enough



*Down the ladder : more cost,  
more signal*

- Four families, in increasing cost and usefulness

- The classic evaluators live in `langchain_classic.evaluation` :

```
from langchain_classic.evaluation import load_evaluator
```

- A real package, not installed with LangChain :

```
pip install langchain-classic
```

# String Evaluators

- The cheapest check is whether two strings say the same thing
- BLEU, ROUGE and METEOR all score overlap, and all score it differently :
  - BLEU counts matching n-grams, and is harsh on short texts
  - ROUGE measures recall against the reference
  - METEOR allows stems and synonyms, so it forgives paraphrase
- Never mix metrics across a comparison ; the numbers are not on one scale
- Exact and regex matching still have their place :
  - Use them wherever the answer has one correct form
- All of them break on a correct answer worded differently :
  - "Warsaw is the capital of Poland" against "The capital of Poland is Warsaw"

# Embedding Distance

- Embedding distance compares meaning instead of wording :

```
evaluator = load_evaluator("embedding_distance", embeddings=make_embeddings())
evaluator.evaluate_strings(prediction=..., reference=...)
```

- Measured against `nomic-embed-text`, with the prediction held constant :

```
prediction : "The capital of Poland is Warsaw"

0.0000 "The capital of Poland is Warsaw"
0.0342 "Poland's capital is named Warsaw"
0.3387 "The capital of Burkina Faso is named Ouagadougou"
```

- The score is a distance, so lower means closer :
  - A paraphrase scores near zero where BLEU would have punished it
- The threshold is ours to pick, and it depends on the embedding model

# Comparison Evaluators

- A comparison evaluator picks the better of two candidate answers :
  - Useful for A/B testing a prompt, a model or a tool description
- `labeled_pairwise_string` compares both against a reference :

```
evaluator = load_evaluator("labeled_pairwise_string")

result = evaluator.evaluate_string_pairs(
 input="What is the capital of Poland?",
 prediction="Warsaw is the capital of Poland",
 prediction_b="I don't know",
 reference="Warsaw is Poland's capital")
```

- The result carries a verdict, a score and the reasoning behind it
- Comparison sidesteps the hardest part of scoring :
  - Choosing between two answers is easier than grading one in isolation

# Criteria Evaluators

- A criteria evaluator scores an answer against a rubric we write :
  - Correctness, conciseness, relevance, helpfulness, harmfulness
- `labeled_criteria` scores against a reference, `criteria` scores without one :

```
evaluator = load_evaluator("labeled_criteria", criteria="correctness", llm=llm)

result = evaluator.evaluate_strings(
 prediction="2 + 2 = 5", input="Calculate 2 + 2", reference="4")

{"reasoning": "...", "value": "N", "score": 0}
```

- Custom criteria are a dictionary of name to description
- Write the rubric before the agent runs, so its output cannot shape the criteria



# LLM as Judge

- Criteria and comparison evaluators are both a model grading a model
- A model judge is the only practical way to score open-ended text at volume
- It also inherits every weakness the model has :
  - Position bias ; the answer shown first tends to win
  - Verbosity bias ; the longer answer tends to look more thorough
  - Self-preference ; a model rates its own style highly
  - The judge can be wrong in exactly the way the agent was wrong
- Three things make it usable :
  - Use a stronger model as the judge than the one being judged
  - Ask for the reasoning, and read it on a sample by hand
  - Calibrate the judge against human labels before trusting the number

# Trajectory Evaluation

- A trajectory evaluator judges the steps as well as the final answer
- It compares the agent's path against a reference path :

```
evaluator = load_evaluator("trajectory", llm=llm)

result = evaluator.invoke({
 "question": "What is the 5th number of the Fibonacci sequence?",
 "answer": "4",
 "agent_trajectory": prediction_steps,
 "reference": {"answer": "5", "agent_trajectory": reference_steps}})
```

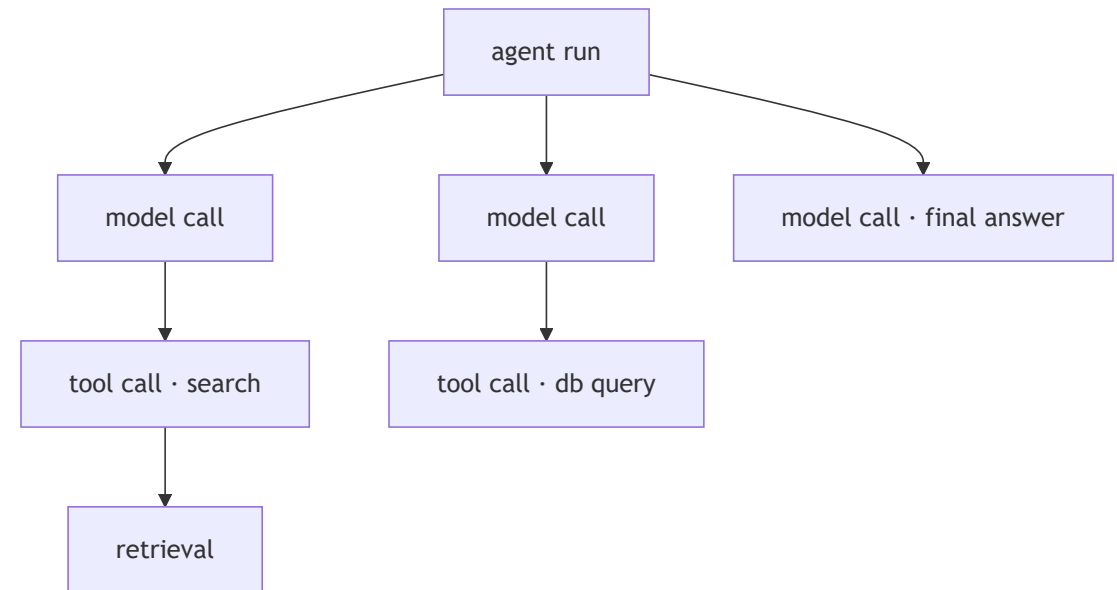
- Trajectory evaluation catches the agent that is right for the wrong reason
- It also catches the reverse :
  - A sound path that failed on one bad tool result is a different bug
  - The answer score alone cannot tell those two apart

# Building an Eval Set

- **An evaluator is worthless without cases to run it on**
- **Take the cases from real traffic :**
  - The inputs we invent are the ones the agent already handles
  - The interesting cases are the ones users sent
- **Start small, at twenty to fifty cases, and grow it from incidents :**
  - Every production failure becomes a case in the set
- **Cover the shapes that break agents :**
  - Ambiguous requests, missing information, out-of-scope questions
  - Inputs where the right answer is to refuse
- **Freeze the set, and version it alongside the code**
- **Run it on every prompt change, because prompts regress silently**

# Observability

- Evaluation says whether the answer was good ; observability says what happened
- One agent run is a tree of nested calls
- For every node we want the same four things :
  - The rendered prompt in, the tool-call arguments out
  - The token counts, the latency, the error if any
- Printing the final message tells us none of that
- The rendered prompt is the most useful field :
  - Most agent bugs are prompt bugs, obvious once it is visible



# Tracing with LangSmith

- LangSmith is the LangChain team's tracing and evaluation platform
- Tracing is switched on with environment variables, before the agent is built :

```
import os

os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_ENDPOINT"] = "https://api.smith.langchain.com"
os.environ["LANGSMITH_API_KEY"] = "<your-langsmith-api-key>"
os.environ["LANGSMITH_PROJECT"] = "running-agents"
```

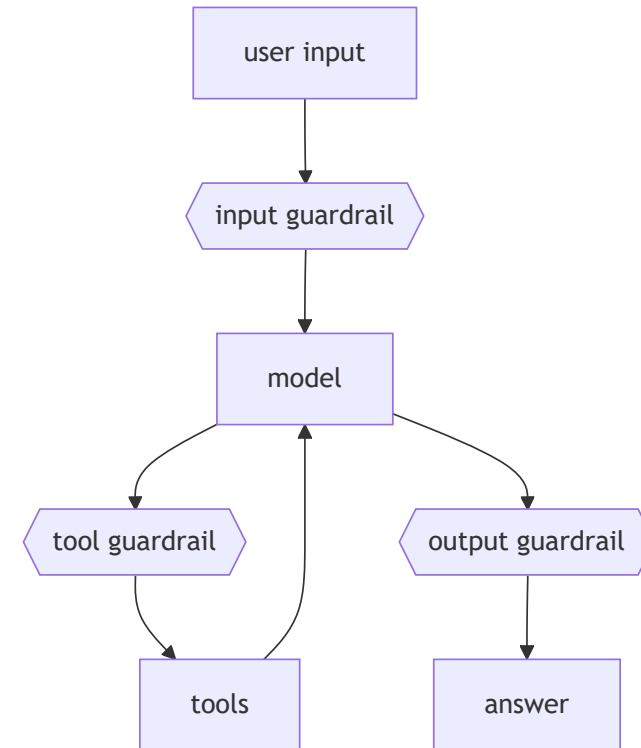
- Nothing else in the code changes, and every run then shows up in the project
- Set them before the agent is built ; tracing is switched on at creation time
- The model still runs locally, and only the trace leaves the machine
- Langfuse, Phoenix and Weights and Biases are the OpenTelemetry alternatives

# Reading a Trace

- **Open a failed run and work down the tree from the root**
- **Read the rendered system prompt first :**
  - Check that the variables were substituted with what we expected
- **Then read the first tool call the model made :**
  - A wrong first call means the tool descriptions are the problem
  - A right first call with a wrong argument means the schema is the problem
- **Then read what the tool returned :**
  - An empty or unhelpful result explains most loops on its own
- **Compare a failing run against a passing one on the same input :**
  - The first node where they diverge is almost always the cause

# Guardrails

- A guardrail constrains what goes in, what comes out, or what may be done
- Three places to put one :
  - On the input, before the model ever sees it
  - On the output, before anybody acts on it
  - Around the tools, on the actions with consequences
- Guardrails are ordinary code, and are deterministic where evaluation is not
- They are also the only part of this module that is a security control :
  - Module 08 covers the tool-side risks in detail
- In LangChain they are middleware, and run around the agent loop



# Input Guardrails

- Validate and sanitise the input before it reaches the model
- `PIIMiddleware` detects and rewrites personal data in place :

```
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(model=llm, tools=[lookup],
 middleware=[PIIMiddleware("email", strategy="redact")])
```

- What the model receives :

```
HumanMessage | My email is [REDACTED_EMAIL], check order AB-9 for me.
ToolMessage | Order AB-9 shipped.
AIMessage | Your order AB-9 has been shipped.
```

- Built-in types are email, credit card, IP address, MAC address and URL
- The strategies are `redact`, `mask`, `hash` and `block`



# Output Guardrails

- Check the output before anything downstream depends on it
- Structural checks need no model and cost nothing :

```
from langchain_classic.evaluation import (
 JsonValidityEvaluator, RegexMatchStringEvaluator)

JsonValidityEvaluator().evaluate_strings(prediction='{x: 1}')
```

```
RegexMatchStringEvaluator().evaluate_strings(
 prediction="Order ID: ABC-1234",
 reference=r"^Order ID: [A-Z]{3}-\d{4}$")
```

```
{'score': 0, 'reasoning': 'Expecting property name enclosed in double quotes'}
{'score': 1}
```

- On a failure, retry with the error text rather than passing it on
- An output parser is already a guardrail, and fails loudly by design

# Human in the Loop

- The strongest guardrail on an irreversible action is a person
- `HumanInTheLoopMiddleware` pauses the agent before a named tool runs :

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

agent = create_agent(model=llm, tools=[refund],
 middleware=[HumanInTheLoopMiddleware(interrupt_on={"refund": True})],
 checkpointer=InMemorySaver())
```

- The run stops and hands back the pending call, its arguments and the choices :

```
out = agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)
```

- The decisions are `approve`, `edit`, `reject` and `respond`
- A checkpointer is required, because the run has to survive the pause

# Cost and Token Budgets

- **Cost in an agent is driven by the loop :**
  - Every pass resends the whole history, so tokens grow with the square of the turns

- **Measure before optimising :**

```
from langchain_core.callbacks import get_usage_metadata_callback

with get_usage_metadata_callback() as cb:
 agent.invoke({"messages": [{"role": "user", "content": question}]})
 print(cb.usage_metadata)
```

- **One question, one tool call, one answer, against qwen3.5:2b :**

```
{'qwen3.5:2b': {'input_tokens': 604, 'output_tokens': 45, 'total_tokens': 649}}
```

- **604 input tokens for a nine-word question, because the schemas travel every call**
- **That number is the four context strategies not being applied**
- **Reasoning tokens are output tokens we pay for and never see :**
  - The same exchange costs 45 output tokens with reasoning off and 163 with it on

# Cost at Volume

- Token counts only mean something once a price and a volume are put against them
- One ticket, measured on the previous slide : 604 input tokens and 45 output
- A service desk handling 10,000 tickets a month, at two illustrative rates :

|             | input      | output      | 10,000 tickets |
|-------------|------------|-------------|----------------|
| small model | 0.50 EUR/M | 1.50 EUR/M  | 3.70 EUR       |
| large model | 3.00 EUR/M | 15.00 EUR/M | 24.90 EUR      |

- Those are single-call figures, and a single call is not what an agent does
- A ten-turn agent resends its history on every pass :
  - Roughly 15,000 input tokens per ticket instead of 604
  - The same 10,000 tickets then cost about 82 EUR, or 519 EUR on the large model
- The model is rarely the expensive decision ; the loop is
- Halving what sits in the window halves the bill

# Caching

- Providers cache a repeated prompt prefix and charge less for the hit
- The prefix is only stable if everything at the front of the prompt is stable :
  - The system prompt, then the tool definitions, then the history
- That makes the tool list a cost decision as well as a design one :
  - Tools added or reordered per request destroy the cache on every call
  - So does a timestamp or a request ID in the system prompt
- Put anything variable at the end of the prompt, never at the front
- Prefix stability is a context decision before it is a cost one
- `SummarizationMiddleware` bounds a long history by compressing older turns :
  - It trades the cache hit on those turns for a much smaller prompt

# Deployment Shapes

- **A request and response service is the common case :**
  - One HTTP call in, one answer out, and the run finishes inside it
  - Agent runs take seconds to minutes, so timeouts need raising
- **A queue and worker suits anything long-running :**
  - The caller gets a job ID and polls, and a retry does not repeat the work
- **Streaming changes what the user sees, and leaves the cost exactly as it was**
- **Whatever the shape, three things change on the way to production :**
  - The checkpointer moves from `InMemorySaver` to a real database
  - Tracing goes on for every run, and stays on
  - Every limit gets a number ; recursion, calls, tokens and timeout

# Writing the Loop by Hand

- An agent is a model, a list of tools and a loop ; none of it needs a framework
- Each request carries the conversation so far and the tool descriptions
- Each reply is one of two shapes : tool calls, or an answer

```
while True:
 message = client.chat.completions.create(
 model=MODEL, messages=messages, tools=SCHEMA).choices[0].message
 messages.append(message)
 if not message.tool_calls:
 return message.content
 for call in message.tool_calls:
 result = TOOLS[call.function.name](**json.loads(call.function.arguments))
 messages.append({"role": "tool", "tool_call_id": call.id, "content": result})
```

- With `list_files`, `read_file`, `write_file` and `run_python` it fixes a failing program in about six turns
- Every coding agent is this loop, with more tools and more care around it

# What a Harness Adds

- **Four things that loop lacks, and each one is a failure from this module :**
  - A turn limit, so a model that never stops asking is stopped
  - Tool errors returned as results, so the model can read them and recover
  - An approval hook that sees every call before it runs
  - A transcript and a token meter, so we can see what it did and what it cost
- **The request grows every turn, because the whole conversation is sent again**
- **The wording of a refusal matters ; told only "not approved", the model finds another route**
- **A one-line hook is a demonstration, and a permission system is the product**
- **Claude Code, Codex and pi add streaming, compaction, sub-agents and MCP on top of the same loop**



# Designing the Tool List

- **The loop did not change between the three tasks the course ends on :**
  - Fix a program whose checks fail ; the tools are files and a Python runner
  - Turn a folder of exports into a report ; the same four tools
  - The service desk of the first capstone ; lookup, reset and escalate
- **What changed was the tool list and the brief, and that is where an agent is designed**
- **A tool the model cannot see is a thing the agent cannot do :**
  - Refuse `write_file` and it diagnoses the bug, then asks us to make the change
  - Remove it altogether and it does not stop ; it guesses at files until the limit does
- **`run_python` is a shell that runs whatever the model wrote, on this machine :**
  - The strongest argument in the course for a person in the loop

# Summary : Running Agents

- Agents fail silently and non-deterministically, so they need measuring
- Evaluators come in four families :
  - String, comparison, criteria and trajectory
- An LLM judge is the only way to score open-ended text, and it has biases
- The eval set matters more than the evaluator, and comes from real traffic
- Observability answers what happened ; read the rendered prompt first
- Guardrails sit on the input, on the output and around the tools
- A human is the right guardrail on an irreversible action
- Bound the loop with limits, and bound the cost with a stable prompt prefix
- An agent is a loop we can write ourselves ; the harness around it is where the work is